On Verifiable Delay Functions from Time-Lock Puzzles

Hamza Abusalah[1], Karen Azari[2]*, Dario Fiore[1], Chethan Kamath[3], and Erkan Tairi[4]

[1] IMDEA Software Institute
{hamza.abusalah,dario.fiore}@imdea.org

[2] University of Vienna, Austria
karen.azari@univie.ac.at

[3] IIT Bombay
ckamath@cse.iitb.ac.in

[4] DIENS, École normale supérieure, CNRS, Inria, PSL University
erkan.tairi@ens.fr

Abstract. A verifiable delay function (VDF) [Boneh et al., CRYPTO 2018] is a function f that is slow-to-compute, but is quickly verifiable given a short proof. This is formalised using two security requirements: i) sequentiality, which requires that a parallel adversary is unable to compute f faster; and ii) soundness, which requires that an adversary is unable to generate a proof that verifies wrong outputs. A time-lock puzzle (TLP), on the other hand, is a puzzle that can be quickly generated, but is slow-to-solve. The security requirement here is sequentiality, which requires that a parallel adversary is unable to solve puzzles faster.

In this paper, we study the relationship between these two timed primitives. Our main result is a construction of "one-time" VDF from TLP using indistinguishability obfuscation (iO) and one-way functions (OWFs), where by "one-time" we mean that sequentiality of the VDF holds only against parallel adversaries that do not preprocess public parameters. Our VDF satisfies several desirable properties. For instance, we achieve perfectly sound and short proofs of $O(\lambda)$ bits, where λ is the security parameter. Moreover, our construction is a *trapdoor* (one-time) VDF that can be easily extended to achieve interesting extra properties (defined in our paper) such as trapdoor-homomorphic and trapdoor-constrained evaluation.

Finally, when combined with the results of Bitansky et al., [ITCS 2016], this yields one-time VDFs from any worst-case non-parallelizing language, iO and OWF. To the best of our knowledge, this is the first such construction that only relies on polynomial security.

* Part of the work was done while the author was affiliated with ETH Zurich, Switzerland.

1 Introduction

Verifiable delay functions (VDFs) are functions that are slow to compute but efficient to verify. Introduced by Boneh et al. [15], it consist of three algorithms: $\mathsf{Setup}(t)$ takes a time parameter t and outputs public parameters pp;[5] $\mathsf{Eval}(\mathsf{pp}, x)$ takes an input x and returns an output y and a short proof π; and $\mathsf{Ver}(\mathsf{pp}, x, y, \pi)$ accepts or rejects a proof π that vouches for y being the output corresponding to x. The "slow to compute but efficient to verify" concept is characterized via three main properties: *efficiency* requires Eval to be computable in sequential time t and Ver in time $\mathrm{polylog}(t)$; *t-sequentiality* ensures that no parallel machine can compute the function in significantly less than t steps; *soundness* means that no efficient adversary can find a proof for an input x that is deemed valid for an output y' different from the correct output y produced by Eval.

An important efficiency measure of VDFs is its tightness. A VDF is *$\delta(t)$-tight* if (y, π) can be computed in (sequential) time $\delta(t)$. Closely related to tightness is *space efficiency* of Eval. It is desired that $\delta(t)$ is as close to t as possible, say $t + O(1)$ or $t + \mathrm{polylog}(t)$, and that the space complexity be constant or poly-logarithmic.

VDFs essentially allow proving that a certain amount of time has elapsed, a property which has been shown useful in a variety of applications like randomness beacons [46, 15, 31, 30], computational timestamping [45], resource-efficient blockchains [25], proofs of replication [4], and even complexity theory [23, 30, 9]. For instance, when generating random beacons from blockchains or through the commit-and-reveal approach, a common problem is that parties may actively manipulate the random source to obtain a desired bias in the resulting beacon. Passing the beacon through a VDF enables introducing a long delay so that the attacker learns too late the impact of its manipulation on the final beacon.

Motivated by these timely applications, a sequence of works has proposed various VDF constructions [15, 52, 62, 27, 28, 31, 30, 47, 14, 39, 22, 37, 9, 33, 38, 2, 57, 41]. These VDFs rely on sequentiality assumptions (e.g., that a certain function is computable only sequentially) and cryptographic assumptions (e.g., the soundness of SNARGs). These two classes of assumptions are somewhat incomparable and play two distinct roles in VDF constructions: the former are used to argue sequentiality, while the latter are typically needed to prove soundness.

Constructions of Delay/Sequential Functions. A function $f : \mathcal{X} \to \mathcal{X}$ is called *t-iterated sequential function* (t-ISF) [15] if it is t-sequential and is of the form

$$f(x) \coloneqq \tau^t(x) \coloneqq \underbrace{\tau(\cdots\tau(\tau(x))\cdots)}_{t \text{ times}} \quad \text{where} \quad \tau : \mathcal{X} \to \mathcal{X}. \tag{1}$$

Sequential functions that do not necessarily have this iterated structure are simply called *sequential functions*.

[5] For simplicity of exposition, we will omit the security parameter in this section and keep explicit only the time parameter.

A number of ISFs are known from ideal and number-theoretic assumptions. Below we assume for simplicity that evaluating $\tau(x)$ takes a single time unit. For example, for a random oracle $\tau(\cdot)$, the function $f(\cdot)$ from (1) is provably t-sequential in the ROM [48], i.e., it can't be computed in $t-1$ RO *rounds* except with negligible probability. Also repeated squarings in groups of unknown order give rise to ISFs. That is, for a group of unknown order $(\mathbb{G},\cdot)$ and $\tau:\mathbb{G}\to\mathbb{G}$ defined as $\tau(x)=x\cdot x$ for $x\in\mathbb{G}$, the function $f(\cdot)$ is assumed to be a t-ISF [53, 52, 62, 40]. Lattice-based ISFs are known based on the iterated application of the binary decomposition operation, followed by a collision-resistant hashing based on the short integer solution (SIS) problem. In particular, let $\tau:\mathbb{Z}_q\to\mathbb{Z}_q$ be such that $\tau(x)=-AG^{-1}(x) \mod q$ where $A\leftarrow\mathbb{Z}_q^{n\times m}$, $m\approx n\log q$ and $G^{-1}:\mathbb{Z}_q^n\to\mathbb{Z}_2^m$ is the binary decomposition operator. Then, $f(\cdot)$ is assumed to be an $o(t)$-ISF [43].[6]

Besides ideal and number-theoretic candidates, ISFs can be constructed *generically* assuming FHE and the (mere) *existence* of ISFs [39], where the latter in turn reduces to the existence of *average-case non-parallelizing languages*.[7]

Constructions of VDFs. All known VDF constructions [15, 52, 62, 27, 28, 31, 30, 47, 14, 39, 22, 37, 9, 33, 38, 2, 57, 41] follow the same design principle: evaluate an (I)SF and generate a proof of correctness based on an appropriate succinct proof system. One can further classify these VDFs based on the generality (or specificity) of the underlying (I)SF and the overlying succinct proof system.

The first category of VDFs is based on *concrete* ISFs and *concrete interactive* succinct proof systems. The VDFs of Pietrzak [52] and Wesolowski [62] are both based on the ISF from repeated squarings in groups of unknown order, and furthermore, both VDFs come with concrete *interactive* succinct proofs of exponentiation, which are then made non-interactive in the ROM by relying on the Fiat-Shamir heuristic [32]. Neither VDF [52, 62] achieves $t+O(1)$ tightness while maintaining a polylog(t) space complexity. However, [52, 62] can be made $(t+O(1))$-tight by applying the result of [28], which, given any $(t+O(t))$-tight VDF *built from any ISF* $f(\cdot)$, transforms it into a $(t+O(1))$-tight VDF. However, this transformation comes at the cost of a multiplicative $\log(t)$ factor on the proof size, verification time and parallelism of Eval.

A lattice-based VDF is obtained by combining the lattice-based interactive succinct proof system [24][8] for SIS relations with the lattice-based ISF [43]. This is made non-interactive in the ROM by relying on the Fiat-Shamir heuristic [32].

Relying on the statistical soundness of Pietrzak's VDF, [9] constructed a variant of Pietrzak's VDF, and then made it non-interactive by instantiating Fiat-Shamir in the standard model by assuming the polynomial hardness of LWE.

[6] Known attacks [51] break bounded-norm variants of $f(\cdot)$, but do not apply to this exact form of $f(\cdot)$.

[7] Due to reliance on universal circuits, this generic construction, however, suffers from an undesirable gap between the honest and adversarial computations of the ISF.

[8] [24] constructs a lattice-based SNARK for NP in the ROM that in particular is efficient for proving SIS relations.

The second category of VDFs is based on *any* (I)SF and *generic* succinct non-interactive arguments (SNARGs) for P. The first line in this category constructs VDFs [15] from any t-ISF with a succinct description[9] and any SNARG for P with quasi-linear time provers. The resulting VDFs are $(t + O(1))$-tight and require $O(\log(t))$ space and $O(\log(t))$ parallelism to compute.[10] The second line in this category constructs $(t + \text{polylog}(t))$-tight VDFs [33] from any t-SF (not necessarily an ISF) and a non-interactive succinct parallelizable argument (SPARG) for P with quasi-linear time provers. The resulting VDF, however, requires $\text{polylog}(t)$ parallelism to compute.

In Section 1.3, we discuss more related work on isogeny-based VDFs and VDFs based on computing roots in finite fields.

Our approach: Time-lock puzzles. As we have seen so far, all known VDFs are constructed from generic/concrete (iterated) sequential functions and succinct proof systems. In this work we put forth a novel design paradigm that breaks free from this blueprint. In particular, we relate VDFs to time-lock puzzles (TLPs) by showing how to construct VDFs from any TLP (by additionally relying on indistinguishability obfuscation, puncturable PRFs and injective one-way functions).

A TLP [54, 11] for a message space $\mathcal{S}$ is a tuple $(\mathsf{PGen}, \mathsf{Sol})$, such that PGen, on input a solution $s \in \mathcal{S}$ and a hardness parameter t, outputs a puzzle z in $\text{polylog}(t)$ (sequential) time, and the puzzle solver, Sol, on input a puzzle z, outputs a solution s in t (sequential) time. The first TLP construction dates back to Rivest, Shamir and Wagner (RSW) [54], who constructed a TLP based on the sequentiality assumption of repeated squaring in groups of unknown order. Bitansky et al. [11] constructed TLPs from (succinct) randomized encodings [3] by additionally assuming the *existence of worst-case non-parallelizing languages* (wcNPL). Informally speaking, for a parameter t, a t-non-parallelizing language is decidable in time t but hard for circuits of depth significantly lower than t. This is arguably a rather minimal complexity-theoretic assumption related to sequentiality. Recently, Bitansky and Garg [10] constructed succinct randomized encodings suitable for TLP constructions assuming circular LWE, and hence, reducing the complexity of constructing TLPs to circular LWE and wcNPL.

Let us, at least informally, highlight the main differences between these two seemingly (un)related timed primitives, namely, VDFs and TLPs. Informally, VDFs and TLPs are located at the extreme ends of the sequential computation asymmetry: a VDF takes time t to compute and time $\text{polylog}(t)$ to verify, while a TLP takes $\text{polylog}(t)$ time to generate a puzzle and t time to retrieve the solution contained in the puzzle. Moreover, while TLPs can be viewed as one-time primitives, VDFs are not. Furthermore, while TLPs do not come with a setup algorithm and their security is not required to hold against pre-processing adversaries, VDFs come with a setup algorithm that outputs public parameters that are used to evaluate the function on all inputs and security must hold with respect to polynomial-time pre-processing of the public parameters.

[9] Therefore, the ROM-based ISF does not fall into this category either.

[10] The constants hidden in the O notation can be as large as 10^5 (see [16, Sect. 5.1]).

Therefore, it is not surprising that these differences translate into different VDF and TLP constructions both in terms of techniques and assumptions.[11] To illustrate, on the one hand, while TLPs can be constructed from wcNPL and succinct randomized encodings [11], no VDF construction is known from the same assumptions/techniques. On the other hand, while we have VDF constructions from general assumptions such as iterated sequential functions and SNARGs with quasi-linear time provers [15], or from sequential functions and SPARGs [33], no TLP constructions are known from these assumptions/techniques. However, despite these differences, some of the early VDF constructions [52, 62] resemble those of the original RSW TLP construction [54], but, as we have just seen, this is the exception rather than the rule. In fact, these VDFs can be seen as adding public verifiability to the otherwise designated-verifier RSW construction.

It is also worth pointing that given a *trapdoor* VDF (TVDF) it is possible to trivially construct a TLP. A TVDF is a keyed function family $\{\mathsf{F}\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}\}$ with a trapdoor-key space $\mathcal{K}$. In a TVDF, there is an additional evaluation algorithm TEval. The Setup algorithm, in addition to outputting pp, samples a uniformly random trapdoor key $k \in \mathcal{K}$ such that $\mathsf{TEval}(k, x)$ running in $\mathrm{polylog}(t)$ sequential time outputs a valid pair (y, π), i.e., y coincides with the output of $\mathsf{Eval}(\mathsf{pp}, x)$ and $\mathsf{Ver}(\mathsf{pp}, x, y, \pi) = 1$. Given a TVDF[12] with output space $\{0,1\}^\lambda$, we can construct a TLP $(\mathsf{PGen}, \mathsf{Sol})$ with solution space $\{0,1\}^\lambda$ as follows: $\mathsf{PGen}(\lambda, t, s)$ runs $(\mathsf{pp}, k) \leftarrow \mathsf{Setup}(\lambda, t)$, samples a random $x \leftarrow \mathcal{X}$, computes $(y, \pi) \leftarrow \mathsf{TEval}(k, x)$ and outputs $(\mathsf{pp}, x, s \oplus y)$; $\mathsf{Sol}(z := (z_0, z_1, z_2))$ computes $(y', \pi') \leftarrow \mathsf{Eval}(z_0, z_1)$ and outputs $y' \oplus z_2$. (Note that the VDF verifiability is not necessary for the implication.)

Given this state of affairs, formally relating TLPs and (plain) VDFs is both challenging and interesting as it advances our understanding of these two central timed primitives and opens a new avenue for their design.

1.1 Our Contributions

Our main result is a VDF construction from any TLP, together with indistinguishability obfuscation, puncturable PRFs and injective one-way functions. This in turn constitutes a first step towards fully understanding how TLPs and VDFs are related.

In a nutshell, our VDF inherits its sequentiality guarantees from those of the TLP and it relies on the cryptographic assumptions to obtain soundness. Our

[11] For completeness, we mention that TLPs based on repeated squarings that achieve advanced features such as homomorphism [50] and batchability [29] are defined with a setup algorithm and are assumed to be secure against pre-processing attacks. However, standard TLPs [54, 11] without advanced features are defined and constructed without setup and their security does not readily extend to the pre-processing setting. In this work, we rely on standard TLPs.

[12] For simplicity, we assume, as we achieve in our construction, that the TVDF has pseudorandom outputs, otherwise, one can rely on the unpredictability of the TVDF output to extract pseudorandom bits.

VDF (a) is the first construction that relates VDFs to TLPs, (b) is *perfectly* sound, (c) achieves $t + \text{polylog}(t)$ tightness, (d) inherits the parallel and space complexities of the underlying TLP, (e) has pseudorandom outputs, (f) can be extended to a trapdoor VDF and (g) allows for *homomorphic* and *constrained* trapdoor keys.

However, the downside of our VDF is that it only achieves a weaker security notion: its sequential security does not hold against pre-processing attacks, i.e., this can be viewed as a *one-time* VDF where time starts kicking once Setup is finished running, that is, the VDF is not re-usable. While for TLPs it is standard to consider a model without preprocessing, in the setting of VDFs this is non-standard and results in a definition of less broad applicability. Generalising our results to a model with preprocessing is a very exciting open question and will most likely require novel ideas (if possible at all).

Nevertheless, compared to the state of the art, our VDF offers the following advantages:

- *New approach:* Our construction departs from known methodologies of building VDFs, which seemed to inherently require sequential functions. We believe that diversifying construction techniques for VDFs could bring new insights towards a better understanding of this relatively novel cryptographic primitive. Additionally, our approach unveils a new connection between two timed-release cryptographic primitives, VDFs and TLPs, which despite their (dis)similarity were not known to be connected.
- *Assumptions:* Combined with the previously mentioned result of Bitansky et al. [11], we obtain the feasibility of VDFs from the existence of iO and OWFs (on the cryptographic side), and wcNPL (on the sequentiality side). Prior to our work, (non-tight) VDFs were known, via adaptively-sound SNARGs for **NP**, from any sequential function, *sub-exponentially* secure iO and *structured* variants of OWF [58, 60]. In contrast, we relax the complexity-theoretic requirement to the bare minimum of wcNPL, and weaken the cryptographic assumptions to polynomially-secure iO and standard OWFs.
- *Perfect soundness:* Our construction provides the only VDF from general assumptions that achieves perfect soundness. Before our work, the strongest notion of soundness for any VDF was statistical soundness, which was achieved by VDFs over (certain) groups of unknown order [52, 14].
- *Tightness:* Our VDF achieves $t+\text{polylog}(t)$ tightness and produces constant-size proofs without any additional parallelsim. Previously known tight VDFs [15, 28, 33] require $\log(t)$ parallelism and their proofs are at least of $\log(t)$ size.
- *Pseudorandom outputs:* Our VDF directly achieves pseudorandom outputs (i.e., they cannot be distinguished from random in time significantly less than t), and this property comes basically for free, without the help of randomness extractors.
- *Trapdoor VDFs:* Our VDF can be extended to a trapdoor VDF (TVDF) [62]. The only known TVDFs are the constructions of VDFs in groups of unknown order [52, 62], and hence, ours is the only *generic* TVDF.

We note that TVDFs can be used to construct *short-lived signatures* [5]. These are signatures that are only valid up to a specified future point in time, after which signatures are forgeable. Using our TVDFs from general assumptions gives rise to the first construction of such signatures from general assumptions.

– *Extensions: trapdoor homomorphic and constrained VDFs:* Our VDFs easily lend themselves to two novel extensions that we introduce: trapdoor-homomorphic and trapdoor-constrained VDFs. We can easily achieve these extensions because the trapdoor evaluation algorithm of our trapdoor VDF constructions is simply identical to the evaluation of a pseudorandom function (PRF). Therefore, instantiating the underlying PRF with a PRF that has extra features, such as key-homomorphic or constrained PRFs, directly gives us these novel trapdoor VDFs. We note that we do not know how to achieve these extensions for existing VDF constructions, such as those based on repeated squaring, due to the lack of homomorphism and constraining capability. Hence, we see this as another benefit of having a novel construction approach for VDFs. In more detail:
  - *Trapdoor-constrained VDFs.* There is an additional constraining algorithm Constr such that on input a trapdoor key k and a subset S from a set of subsets $\mathcal{S}$, Constr outputs a constrained key k_S that allows evaluating the VDF on inputs $x \in S$ fast, i.e., in polylog(t) time. By plugging in our trapdoor-constrained VDF in the short-lived signatures construction [5], we directly get *constrained short-lived signatures* where signing keys can be constrained to sign a subset of messages. The concept of constrained short-lived signatures was not defined/constructed before.
  - *Trapdoor-homomorphic VDFs.* There is a homomorphism from the trapdoor key space to the output space. Let $(\mathcal{K}, \oplus)$ and $(\mathcal{Y}, \otimes)$ be groups, $i \in \{0, 1\}$, then for trapdoor keys k_i and evaluations $(y_i, \pi_i) \leftarrow \mathsf{TEval}(k_i, x)$ and $(y, \pi) \leftarrow \mathsf{TEval}(k_0 \oplus k_1, x)$, it holds that $y_0 \otimes y_1 = y$. To the best of our knowledge no other TVDF is trapdoor-key homomorphic.

As an additional technical contribution, we construct prefix-constrained signatures, which we need for the security proof of our main construction, using one-time signatures and constrained PRFs. Our construction of prefix-constrained signatures is in fact the same as the puncturable signature scheme of [21], but we need to rely on constrained PRFs for the constraint class of "dual" prefixes and arbitrary-length inputs, which can be obtained by minor modification of the PRF construction of Goldreich, Goldwasser and Micali (GGM) [35] based on OWFs. We believe that the construction of constrained signatures for larger constraint sets can be of independent interest.

1.2 Technical Overview

For this overview, we focus on constructing VDF from TLP, iO and injective one-way function (IOWF). Our main result follows thanks to prior works which showed how to construct (1) TLP from non-parallelising languages and iO [11] and (2) IOWF from (polynomially-secure) iO and OWF [13].

VDF from TLP: seems straightforward? Let's first recall the definition of TLPs. It consists of two algorithms: a randomised puzzle generator algorithm PGen that, on input a solution s and a hardness parameter t, outputs a puzzle z; and a puzzle solver Sol that, on input a puzzle z, outputs a solution s. Sequentiality requires it to be hard for parallel adversaries of depth "much less than t" to solve z and obtain s (even for worst-case solutions).

Let us first focus on constructing a plain VDF for some fixed hardness parameter t. The construction proceeds in two stages: we first describe how to use a TLP and iO to obtain just a delay function (DF), and then add verifiability to this construction using a one-way permutation. A first attempt would be to mimic the already-discussed blueprint of combining a sequential function, which a TLP implies, with a succinct non-interactive argument (SNARG). However, note that a TLP only allows sampling a puzzle by first sampling the solution – it is not clear how to publicly sample just a puzzle as required for a VDF.[13] To resolve this issue, we rely on indistinguishability obfuscation (iO): the public puzzle sampler consists of an obfuscation of the TLP puzzle sampler. However, since TLP puzzle generation is a randomised procedure, we need to somehow *derandomise* it.[14] To this end, we employ a puncturable pseudorandom function (PPRF) [56]. Recall that a PPRF F is a PRF that additionally supports *puncturing* of any key k at any input x^*, such that: 1) the punctured key k^* preserves functionality on non-punctured inputs $x \neq x^*$; and 2) the evaluation of the PRF at x^* is pseudorandom even given k^*. We use PPRF twice, first to generate an unpredictable solution s from the DF input x, and then to sample pseudorandom coins for the TLP puzzle sampler. Thus, the public parameters of the DF consist of an obfuscation of the following Sample circuit, where k_1 and k_2 are two hardwired PPRF keys.

$z \coloneqq \mathsf{Sample}_{[k_1,k_2]}(x)$

1. Generate a pseudorandom TLP solution $s \coloneqq F_{k_1}(x)$
2. Use s and pseudorandom coins to generate a TLP puzzle $z \coloneqq \mathsf{PGen}(s, t; F_{k_2}(x))$
3. Output z

Thus, to evaluate the DF on an input x, first use the obfuscated Sample circuit (available as part of the public parameter) to generate a puzzle z, and then run the TLP solver to compute the solution s, which is defined to be the output of the VDF. Note that thanks to perfect TLP correctness, a unique solution s is guaranteed.

[13] In fact, a TLP can be thought of as a "designated verifier" VDF, where the verifier is given the TLP solution.

[14] One could use instead *probabilistic* iO, but presently we only know how to construct it from sub-exponentially secure iO [20].

Arguing sequentiality via punctured programming. Recall that sequentiality of a (V)DF requires it to be hard for parallel adversaries of depth "much less than t" to compute the output on random inputs. The sequentiality of the DF described above can be reduced to sequentiality of the underlying TLP using two applications of the punctured programming [56], as explained below.

- First, we *undo* the derandomisation in Step 1 by switching the solution corresponding to x^* from $F_{k_1}(x^*)$ to a (truly) random s^*. To this end, we puncture k_1 at a randomly *pre-sampled* VDF challenge x^* and program Sample to output s^* when the input is x^*.[15] This switch is computationally indistinguishable due to iO security, preservation of PPRF functionality at non-punctured points and pseudorandomness of PPRF at x^*. Crucially, the pseudorandomness of PPRF holds even given the punctured key k_1^*, which is required to generate the obfuscated Sample circuit.
- Then, we undo the derandomisation in Step 2 by switching the coins used to generate the challenge puzzle z^* from pseudorandom $F_{k_2}(x^*)$ to a random r^*. To this end, we puncture k_2, also at x^*, and program Sample to use r^* when computing the challenge puzzle z^*. This switch is computationally indistinguishable for the same reasons as above.
- At this point, the distribution of z^* is identical to the distribution of TLP puzzle generator and, more importantly, the only use of s^* is to generate z^*. Thus it is possible to invoke TLP sequentiality to argue sequentiality of the DF.

Note that for the whole approach to work it is crucial to be able to pre-sample the (V)DF challenge x^*, which is required to generate the punctured keys hardwired in the obfuscation of Sample in the public parameter. However, this is possible in the VDF sequentiality game.

To see this, let's recall the VDF sequentiality game in a bit more detail. The game works in two phases: First, the adversary receives the public parameters and might do some polynomial-time preprocessing on them. In the second phase, the adversary receives a uniformly random challenge input x^* and is then restricted to depth "much less than t". When simulating the security game, a reduction can clearly sample the challenge x^* already with the public parameters, however must not release it to the adversary before the second phase.

The reason why we only achieve sequentiality *without preprocessing* is that we also hardwire the puzzle z^* in the obfuscation of Sample. A preprocessing adversary, as in standard VDF definitions, would receive the public parameters (i.e., the obfuscated circuit containing the TLP puzzle) and could now do polynomial but otherwise sequentially unrestricted computations on it. Since sequentiality of the TLP only holds against adversaries of restricted sequential computation, it is unclear how to prove security in that adversarial model. We refer to the end of this section for further discussion.

[15] Note that the derandomisation is undone *only* for the "$x = x^*$ branch". This is crucial since the corresponding solution s^* needs to be hardwired into (modified) Sample, and thus we cannot afford to derandomise solution generation for all inputs.

Adding verifiability, with perfect soundness. To add verifiability to the DF described above, an obvious solution is to rely on SNARGs, which are implied by iO and OWF, e.g., as constructed in [56]. However, this approach runs into several problems. First, the soundness requirement of VDF is *adaptive* in nature: given the public parameters, the adversary needs to come up with a proof that attests to a wrong evaluation for a domain point of *its choice*. Most constructions of SNARGs from iO only satisfy *selective* soundness, and the recent works that achieve adaptive soundness [59, 61], do it at the cost of relying on sub-exponentially secure iO (and other additional assumptions). Moreover, the soundness will be inherited from the SNARGs, and therefore, will only be computational. On the other hand, our VDF achieves perfect soundness.

To resolve this, we delegate the proof generation from Eval to the obfuscated circuit in the public parameter. In more details, instead of the standard approach where the VDF evaluation algorithm generates a (succinct) proof while evaluating the VDF, our proof generation is delegated to the obfuscated Sample circuit. (Thus, the VDF evaluation algorithm outputs empty proofs.) To this end, Sample is altered to additionally output a "proof" $f(s)$ of the solution s, where f is an injective one-way function. Hence, the public parameter now consists of an obfuscation of the following Sample' circuit (with differences from Sample highlighted in red).

$(z, \pi) := \mathsf{Sample}'_{[k_1,k_2]}(x)$

1. Generate a pseudorandom TLP solution $s := F_{k_1}(x)$
2. Use s and pseudorandom coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; F_{k_2}(x))$
3. Output $(z, \pi := f(s))$

Consequently, to verify that the value s generated by the evaluation algorithm on input x is indeed the correct output, it suffices to:

- re-run the obfuscated Sample circuit on x to obtain the proof π; and
- check if $f(s) = \pi$ holds.

Assuming perfect correctness of iO and TLP, for an adversary's output (x, s^*), such that $s^* \neq s := F_{k_1}(x)$, to be accepted, it must be that $f(s^*) = \pi = f(s)$. However, this contradicts f's injectivity.

Finally, note that although the proof leaks information about the solution s, we show that it does not affect sequentiality thanks to f's one-wayness. We refer the reader to Section 3 for the technical details.

Achieving advanced features. Building up on the basic construction described above, we obtain VDFs with additional features:

- Obtaining a *trapdoor* VDF (TVDF) is straightforward: the PPRF key k_1 can be used to directly compute the output using PPRF evaluation algorithm.

– Due to the fact that the trapdoor evaluation algorithm of our basic trapdoor VDF construction is just a PRF evaluation function, we can easily obtain trapdoor VDFs with more advanced features, by replacing the underlying puncturable PRF with a PRF with additional features. More precisely, we obtain trapdoor-homomorphic VDFs by using a key-homomorphic puncturable PRF [6, 19] and trapdoor-constrained VDFs by using a constrained PRF [17, 18, 42]. We refer to Section 5 for the definitions and instantiations of these advanced VDFs.
– The basic construction requires the verifier to re-run the obfuscated Sample circuit to obtain a "proof". Evaluating obfuscated circuits can be costly for resource-constrained verifiers, and to remedy this we provide an alternative construction where the verifier only needs to verify a digital signature. The construction is obtained by augmenting Sample to additionally output a digital signature σ as shown below, where sk denotes the signing key of the digital signature.

$(z, \pi, \sigma) := \mathsf{Sample}'_{[k_1,k_2,sk]}(x)$

1. Generate a pseudorandom TLP solution $s := F_{k_1}(x)$
2. Use s and pseudorandom coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; F_{k_2}(x))$
3. Generate signature $\sigma := \mathsf{Sign}(\mathsf{sk}, x \| f(s) \| z)$
4. Output $(z, f(s), \sigma)$

For the proof of soundness to go through, we need to rely on digital signatures with extra structure, namely a *constrained signature scheme* (a.k.a. policy-based signatures) [7] for a particular class of prefix constraints. The soundness of the resulting scheme is only *selective* though. We refer the reader to Section 4 for more details.
– We view our construction of *prefix-constrained signatures* as a contribution of independent interest. Our construction relies on a previous construction of puncturable signatures [21], and is based on one-time signatures (OTS) [44] and constrained PRFs [17, 18, 42]. The construction of [21] is based on a binary tree structure, similar to the paradigm for constructing (many-time) signatures from OTS, where each node in the tree is associated with a pseudorandom seed (generated through a puncturable PRF) that is used to derive a OTS key pair. In order to sign a message m, given the secret seed associated to the root of the tree, one generates all signing keys associated with nodes on the path from the root to the leaf associated with string m and then uses these keys to compute signatures signing the public keys associated to the two children of a node under the secret key associated to that node. To prove security of this construction, [21] rely on security of the underlying PPRF for *variable-length* inputs as well as the one-time security of the OTS. For our construction of *prefix-constrained* signatures, we similarly rely on *constrained* PRFs for the class of "dual" prefixes and OTS. We instantiate

these underlying primitives from OWFs by using Lamport's OTS scheme, and a minor modification of the GGM PPRF [35] from PRGs. This GGM PPRF modification supports variable-length inputs (note, the plain GGM construction is *insecure* for variable-length inputs) and (similar to the GGM PPRF) can be used as a constrained PRF for the required constraint class. This gives us prefix-constrained signatures from the minimal assumption of OWFs.

Open problem: Achieving VDFs with preprocessing. As mentioned above, our VDF construction achieves perfect soundness and can be instantiated to achieve advanced functionalities. On the downside, we only achieve sequentiality in a security model without preprocessing. This means, when applying our VDF construction one needs fresh parameters for each evaluation, opposed to reusing public parameters and only sampling the uniformly random input to the VDF freshly. The issue here is that in our proof, in order to reduce to the sequentiality of TLP, we embed a challenge puzzle in the public parameters, and hence, commit to this puzzle ahead of time. Also when considering a notion of TLP with preprocessing, this issue remains. Even with equivocation techniques there does not seem to be an obvious solution and we believe it will require novel ideas to solve the problem (see Remark 3).

We view this somewhat unexpected difficulty to achieve full-fledged VDF from TLP as an interesting observation that points out that the two notions are probably more different than one might expect at first sight. We hope this will encourage further research on timed primitives from generic assumptions.

1.3 Additional Related Work

A number of isogeny-based VDFs are proposed in [27, 22, 2], however, these schemes fall short of satisfying the standard VDF definition. In particular, sampling a VDF instance [27] is as inefficient as the function evaluation itself, and furthermore, the function instance description is as large as the time parameter itself. For example, let $t = 2^{40}$, then it takes roughly 2^{40} sequential steps to generate a VDF instance for time parameter t, and the instance's size is in the order of 2^{40} bits. On the other hand, the VDFs of [22, 2] suffer from inefficient sampling: sampling a domain element is as (in)efficient as evaluating the VDF.

The VDF constructions given in [57, 41] and the "sloth++" VDF of [15] are built from ISFs $f(\cdot) = \tau^t(\cdot)$, where τ involves computing roots in prime-order finite fields, or extensions thereof. The root computation's sequentiality is put into scrutiny recently. In particular, [8] shows how to speed up root computation beyond their conjectured security. It is instructive to mention that [8] does not find shortcuts in the computation of $f(\cdot)$, i.e., their attacks still evaluate $\tau(\cdot)$ in t steps sequentially, but the individual evaluation of $\tau(\cdot)$ is sped up.

Lastly, we mention additional works that study various feasibility aspects of verifiable delay functions. Mahmoody et al. [49] study constructions of VDFs based only on random oracles and prove that in this model it is impossible to

construct VDFs with perfect soundness and/or tight proof generation. Impossibility of tight VDFs in the random oracle model is proven also by Döttling et al. [28]. Rotem et al. [55] study group-based constructions of (verifiable) delay functions and prove that it is unlikely to build such schemes using generic-group operations in cyclic groups of known order.

A Related Talk. After completing this work, and thanks to an anonymous TCC'25 reviewer, we became aware of a talk [12] at MIT VDF Day 2019 that presents a VDF construction from (the existence of) NPL and iO. Although our base construction and their work differ in techniques, they rely on the same assumptions: while we directly rely on a TLP plus iO, they rely on NPL plus iO, which together imply a TLP (OWFs are implied by TLPs [11]). However, to the best of our knowledge, their work is neither published nor available online in a written form. Our work shows that, even with these strong tools, it is still open to actually construct a full fledged VDF from TLP and iO, and it is even non-trivial to prove one-time security. Moreover, we also define and construct VDFs with additional features, such as trapdoor-constrained and trapdoor-homomorphic VDFs, which were not considered prior to our work.

2 Preliminaries

In this section, we first set up our notation. Then we describe the computational model that we use in this paper. Finally, we define the cryptographic primitives that we rely on, namely one-time signatures, indistinguishability obfuscation, constrained PRFs, prefix-constrained signatures, time-lock puzzles and verifiable delay functions.

Notation. For $n \in \mathbb{N}$, let $[n] := \{1, \ldots, n\}$, and let $\lambda \in \mathbb{N}$ be the security parameter. For a finite set $\mathcal{S}$, we denote by $s \leftarrow \mathcal{S}$ the process of sampling s uniformly from $\mathcal{S}$. Let $y \leftarrow \mathsf{A}(\lambda, x)$ be the process of running a *randomised* algorithm A on input (λ, x) with access to uniformly random coins and assigning the result to y (we may omit to mention the input λ explicitly and assume that all algorithms take λ as input). To make the random coins r explicit, we write $\mathsf{A}(\lambda, x; r)$. We use $y := \mathsf{A}(x)$ to denote a *deterministic* algorithm A outputting y on input x. We use $\perp$ to indicate that an algorithm terminates with an error and A^{B} when A has oracle access to B, where B may return $\top$ as a distinguished special symbol. We say an algorithm A is probabilistic polynomial time (PPT) if the running time of A is polynomial in λ. If an algorithm A runs in polynomial time, but it is deterministic, then we say that it is deterministic polynomial time (DPT). A function f is negligible if its absolute value is smaller than the inverse of any polynomial (i.e., if $\forall c \exists k_0 \forall \lambda \geq k_0 : |f(\lambda)| < 1/\lambda^c$). We denote the class of negligible functions by negl, and hence, we write $f \in \mathsf{negl}$ or $f(\lambda) \in \mathsf{negl}(\lambda)$ if f is a negligible function. We write $q = q(\lambda)$ if we mean that the value q depends on λ.

Computational Model. We follow the convention from [11] of modelling honest algorithms as uniform machines, and adversaries as non-uniform machines.

In particular, an efficient adversary A is modelled as a family of polynomial-sized circuits $\mathsf{A} = (\mathsf{A}_\lambda)_{\lambda\in\mathbb{N}}$. For timed cryptographic primitives, we need to consider parallel adversaries, which are modelled by a family of polynomial-sized, depth-bounded circuits. For a circuit C, we use $\mathit{depth}(\mathsf{C})$ to denote its depth and $|\mathsf{C}|$ to denote its size. Thus, the overall running time of a circuit is determined by $|\mathsf{C}|$, whereas its parallel time is determined by $\mathit{depth}(\mathsf{C})$.

2.1 Basic Cryptographic Primitives

One-Time Signatures. First, we define one-time signatures, which are signatures that allow to use a key pair to sign a single (arbitrary) message, and offer no security guarantees if the same key pair is used to sign a second message.

Definition 1 (One-Time Signatures). *A one-time signature (OTS) scheme* OTS *for a message space* $\mathcal{M} := \left(\{0,1\}^{\ell(\lambda)}\right)_{\lambda\in\mathbb{N}}$*, where* $\ell(\cdot)$ *is a polynomial, is a tuple of algorithms* $(\mathsf{KGen}, \mathsf{Sign}, \mathsf{Ver})$ *with the following syntax:*

- $(\mathsf{msk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$*: On input a (unary represented) security parameter* λ*, the* key-generation *algorithm outputs a master signing-verification key pair* $(\mathsf{msk}, \mathsf{pk})$*.*
- $\sigma := \mathsf{Sign}(\mathsf{msk}, m)$*: On input a (master) signing key* sk *and a message* $m \in \{0,1\}^{\ell(\lambda)}$*, the* signing *algorithm outputs a signature* σ*.*
- $b := \mathsf{Ver}(\mathsf{pk}, m, \sigma)$*: On input a verification key* pk*, a message* m *and a signature* σ*, the* verification *algorithm outputs a bit* b *indicating accept* (1) *and reject* (0)*.*

We require OTS *to satisfy the following properties:*

- Correctness. *For all* $\lambda \in \mathbb{N}$ *and every* $m \in \{0,1\}^{\ell(\lambda)}$*, the following holds:*

$$\Pr\left[\mathsf{Ver}(\mathsf{pk}, m, \sigma) = 1 : \begin{array}{l}(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)\\ \sigma := \mathsf{Sign}(\mathsf{sk}, m)\end{array}\right] = 1.$$

- One-time existential unforgeability. *For every efficient two-stage adversary* $\mathsf{A} = (\mathsf{A}_{0,\lambda}, \mathsf{A}_{0,\lambda})_{\lambda\in\mathbb{N}}$*, we have that*

$$\Pr\left[\begin{array}{r}\mathsf{Ver}(\mathsf{pk}, m_\mathsf{A}, \sigma_\mathsf{A}) = 1\\ m_\mathsf{A} \neq m\end{array} : \begin{array}{l}(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)\\ (m, \mathsf{st}) \leftarrow \mathsf{A}_{0,\lambda}(\mathsf{pk})\\ (m_\mathsf{A}, \sigma_\mathsf{A}) \leftarrow \mathsf{A}_{0,\lambda}(\mathsf{st}, \mathsf{Sign}(\mathsf{sk}, m))\end{array}\right] \in \mathsf{negl}(\lambda),$$

Indistinguishability Obfuscation. We recall the syntax and requirements of indistinguishability obfuscation (iO). Intuitively, iO requires that for any two circuits C_0 and C_1 of the same size that are functionally equivalent, that is $C_0(x) = C_1(x)$ for all inputs x, we have that obfuscations $\mathsf{iO}(C_0)$ and $\mathsf{iO}(C_1)$ are computationally indistinguishable.

Definition 2 (Indistinguishability Obfuscator [34]). *A PPT algorithm* iO *is an indistinguishability obfuscator (iO) for a circuit class* $\{\mathcal{C}_\lambda\}_{\lambda\in\mathbb{N}}$ *if it satisfies the following conditions:*

– Functionality. *For every security parameter* $\lambda \in \mathbb{N}$*, every circuit* $C_\lambda : \{0,1\}^{n(\lambda)} \to \{0,1\}^{m(\lambda)} \in \mathcal{C}_\lambda$ *(where* $m(\cdot)$ *and* $n(\cdot)$ *are polynomials), and every input* $x \in \{0,1\}^{n(\lambda)}$*, we have that*

$$\Pr\left[\tilde{C}_\lambda(x) = C_\lambda(x) : \ \tilde{C}_\lambda \leftarrow \mathsf{iO}(C_\lambda)\right] = 1.$$

– Indistinguishability (security). *For efficient distinguisher* $\mathsf{D} = (\mathsf{D}_\lambda)_{\lambda\in\mathbb{N}}$ *and for any pair of circuits* $C_{0,\lambda}, C_{1,\lambda} \in \mathcal{C}_\lambda$*, such that for all input* $x \in \{0,1\}^{n(\lambda)}$*,* $C_{0,\lambda}(x) = C_{1,\lambda}(x)$ *and* $|C_{0,\lambda}| = |C_{1,\lambda}|$*, it holds that*

$$\left|\Pr\left[\mathsf{D}_\lambda(\mathsf{iO}(C_{0,\lambda})) = 1\right] - \Pr\left[\mathsf{D}_\lambda(\mathsf{iO}(C_{1,\lambda})) = 1\right]\right| \in \mathsf{negl}(\lambda).$$

2.2 Constrained Cryptographic Primitives

Constrained PRFs. Constraint PRFs (CPRFs), introduced by Boneh and Waters [17], are PRFs for which a constraint key for some set S can be given out, that allow evaluation of the PRF on all inputs belonging to S and nowhere else.

Definition 3 (Constrained Functions). *A function family* $\mathcal{F}_\lambda = \{F\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}\}$ *with key space* $\mathcal{K}$*, input space* $\mathcal{X}$ *and output space* $\mathcal{Y}$ *is* constrained *for a family of subsets* $\mathcal{S}_\lambda$ *over* $\mathcal{X}$ *with a* constraint *key space* $\mathcal{K}_\mathcal{S}$ *such that* $\mathcal{K}_\mathcal{S} \cap \mathcal{K} = \emptyset$ *if there exists a tuple of PPT algorithms* $(\mathsf{Gen}, \mathsf{Eval}, \mathsf{Constr})$ *such that*

– $k \leftarrow \mathsf{Gen}(1^\lambda)$: *On input a security parameter* $\lambda \in \mathbb{N}$ *in unary,* Gen *outputs a key* $k \in \mathcal{K}$*.*
– $k_S \leftarrow \mathsf{Constr}(k, S)$: *On input a key* $k \in \mathcal{K}$ *and* $S \in \mathcal{S}_\lambda$*,* Constr *outputs a constraint key* $k_S \in \mathcal{K}_\mathcal{S}$*.*
– $y := \mathsf{Eval}(k, x)$*: On input a key* $k \in \mathcal{K} \cup \mathcal{K}_\mathcal{S}$ *and input* $x \in \mathcal{X}$*,* Eval *computes* $y = F(k, x)$ *if either* $k \in \mathcal{K}$ *or* $k \in \mathcal{K}_\mathcal{S}$ *and* $x \in S$*. (Otherwise* Eval *outputs* $y = \perp$*.)*

Definition 4 (Constrained PRFs [17, 18, 42]). *A constrained function family* $\mathcal{F}_\lambda = \{F\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}\}$ *is a family of* selectively-secure constrained PRFs *if for every PPT stateful adversary* A*, there exists a function* $\delta \in \mathsf{negl}$ *such that*

$$\Pr\left[\mathsf{Exp}^{\mathsf{CPRF}}_{\mathcal{F},\mathsf{A}}(\lambda) = 1\right] \leq \frac{1}{2} + \delta(\lambda),$$

where the experiment $\mathsf{Exp}^{\mathsf{CPRF}}_{\mathcal{F},\mathsf{A}}(\lambda)$ *is defined below.*

$\mathsf{Exp}^{\mathsf{CPRF}}_{\mathcal{F},\mathsf{A}}(\lambda)$

$b \leftarrow \{0,1\}$
$k \leftarrow \mathsf{Gen}(1^\lambda)$
$x^* \leftarrow \mathsf{A}(1^\lambda)$
if $b = 0$ **then**
  $y := \mathsf{Eval}(k, x^*)$
else
  $y \leftarrow \mathcal{Y}$
$b' \leftarrow \mathsf{A}^{\mathcal{O}_{\mathsf{Eval}}, \mathcal{O}_{\mathsf{Constr}}}(y)$
return $b = b'$

$\mathcal{O}_{\mathsf{Eval}}(x)$

if $x \notin \mathcal{X} \vee x = x^*$ **then**
  return $\bot$
return $\mathsf{Eval}(k, x)$

$\mathcal{O}_{\mathsf{Constr}}(S)$

if $S \notin \mathcal{S}_\lambda \vee x^* \in S$ **then**
  return $\bot$
return $\mathsf{Constr}(k, S)$

Puncturable Pseudorandom Functions. Puncturable PRFs (PPRFs), introduced by Sahai and Waters [56], are PRFs for which a key can be given out, such that it allows evaluation of the PRF on all inputs, except for a designated polynomial-size set of inputs.

Definition 5 (Puncturable PRFs [56]). *A constrained PRF family* $\mathcal{F}_\lambda = \{\mathsf{F}\colon \mathcal{K} \times \{0,1\}^n \to \mathcal{Y}\}$ *where* $n = n(\lambda)$ *is a* puncturable PRF *family if it is constrained for sets* $\{0,1\}^n \setminus T$ *for* $\mathrm{poly}(n)$*-sized* $T \subseteq \{0,1\}^n$.

Prefix-Constrained Digital Signatures. Next, we describe prefix-constrained digital signatures, which are constrained digital signatures (a.k.a. policy-based signatures) [7] where the constraint function is restricted to a particular "dual" prefix relation. Our definition is adapted from [36], and is modified to suit our purpose. In the following definition we denote the prefix relation by "$\prec$", i.e., for two strings m, m' we write $m \preceq m'$ if m is a prefix of m', and $m \prec m'$ if m is a *strict* prefix of m'.

Definition 6 (Prefix-Constrained Signatures). *A prefix-constrained signature (PCS) scheme* PCS *for a message space* $\mathcal{M} := \left(\{0,1\}^{\ell(\lambda)}\right)_{\lambda\in\mathbb{N}}$*, where* $\ell(\cdot)$ *is a polynomial, is a tuple of algorithms* $(\mathsf{KGen}, \mathsf{CGen}, \mathsf{Sign}, \mathsf{CSign}, \mathsf{Ver})$ *with the following syntax:*

- $(\mathsf{msk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$*: On input a (unary represented) security parameter* λ*, the* key-generation *algorithm outputs a master signing-verification key pair* $(\mathsf{msk}, \mathsf{pk})$.
- $\mathsf{sk}^* \leftarrow \mathsf{CGen}(\mathsf{msk}, m^*)$*: On input a master secret key* msk *and a prefix* $m^* \in \{0,1\}^{<\ell(\lambda)}$*, the* constrained key-generation *algorithm outputs a constrained signing key* sk^*.
- $\sigma := \mathsf{Sign}(\mathsf{msk}, m)$*: On input a (master) signing key* sk *and a message* $m \in \{0,1\}^{\ell(\lambda)}$*, the* signing *algorithm outputs a signature* σ.
- $\sigma := \mathsf{CSign}(\mathsf{sk}^*, m)$*: On input a secret key* sk^* *constrained on* $m^* \in \{0,1\}^{<\ell(\lambda)}$ *and a message* $m \in \{0,1\}^{\ell(\lambda)}$*, the* constrained-signing *algorithm outputs a signature* σ *for* m.

– $b := \mathsf{Ver}(\mathsf{pk}, m, \sigma)$*: On input a verification key* pk*, a message* m *and a signature* σ*, the* verification *algorithm outputs a bit* b *indicating accept (1) and reject (0).*

We require PCS *to satisfy the following properties:*

– Correctness of signing using master signing key. *For all* $\lambda \in \mathbb{N}$ *and every* $m \in \{0,1\}^{\ell(\lambda)}$*, the following holds:*

$$\Pr\left[\mathsf{Ver}(\mathsf{pk}, m, \sigma) = 1 : \begin{array}{c}(\mathsf{msk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda) \\ \sigma := \mathsf{Sign}(\mathsf{msk}, m)\end{array}\right] = 1.$$

– Consistency of signing using constrained signing key. *For all* $\lambda \in \mathbb{N}$*,* $m^* \in \{0,1\}^{<\ell(\lambda)}$ *and* $m \in \{0,1\}^{\ell(\lambda)}$ *such that* m^* *is* not *a prefix of* m*, the following holds:*

$$\Pr\left[\sigma^* = \sigma : \begin{array}{c}(\mathsf{msk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda) \\ \mathsf{sk}^* \leftarrow \mathsf{CGen}(\mathsf{msk}, m^*) \\ \sigma^* := \mathsf{CSign}(\mathsf{sk}^*, m) \\ \sigma := \mathsf{Sign}(\mathsf{msk}, m)\end{array}\right] = 1.$$

– Selective one-time constrained unforgeability under chosen-message attack. *For every efficient two-stage adversary* $\mathsf{A} = (\mathsf{A}_{0,\lambda}, \mathsf{A}_{1,\lambda})_{\lambda\in\mathbb{N}}$*, we have that*

$$\Pr\left[\begin{array}{rl}\mathsf{Ver}(\mathsf{pk}, m_\mathsf{A}, \sigma_\mathsf{A}) = 1 & (m^*, \mathsf{st}) \leftarrow \mathsf{A}_{0,\lambda} \\ m^* \in \{0,1\}^{<\ell(\lambda)}, m_\mathsf{A} \in \{0,1\}^{\ell(\lambda)} & (\mathsf{msk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda) \\ m^* \prec m_\mathsf{A} & : \mathsf{sk}^* \leftarrow \mathsf{CGen}(\mathsf{msk}, m^*) \\ m_\mathsf{A} \textit{ not queried to } \mathsf{Sign}(\mathsf{msk}, \cdot) & (m_\mathsf{A}, \sigma_\mathsf{A}) \leftarrow \mathsf{A}_{1,\lambda}^{\mathsf{Sign}(\mathsf{msk},\cdot)}(\mathsf{st}, \mathsf{pk}, \mathsf{sk}^*)\end{array}\right] \in \mathsf{negl}(\lambda).$$

2.3 Timed Cryptographic Primitives

Time-Lock Puzzles. We define the syntax and requirements of time-lock puzzles, which are cryptographic puzzles where the solution s remains hidden from adversaries that run in time significantly less than t, including parallel adversaries with polynomially many processors.

Definition 7 (Time-Lock Puzzles [11]). *A time-lock puzzle* TLP *is a pair of algorithms* $(\mathsf{PGen}, \mathsf{Sol})$ *with the following syntax:*

– $z \leftarrow \mathsf{PGen}(t, s)$*: On input a time parameter* t *and a solution* $s \in \{0,1\}^\lambda$*, the* puzzle generation *algorithm outputs a puzzle* $z \in \{0,1\}^{w(\lambda)}$*, for a polynomial* $w(\cdot)$*.*
– $s := \mathsf{Sol}(z)$*: On input a puzzle* z*, the* solver *algorithm outputs a solution* s*.*

We require TLP *to satisfy the following properties:*

– Completeness. *For every security parameter* $\lambda \in \mathbb{N}$*, time parameter* t *and solution* $s \in \{0,1\}^\lambda$*, we have that*

$$\Pr\left[\mathsf{Sol}(\mathsf{PGen}(t, s)) = s\right] = 1.$$

– Efficiency. *We require the following efficiency guarantees:*
  - $\mathsf{PGen}(t, s)$ *can be computed in time* $\mathrm{poly}(\log t, \lambda)$.
  - $\mathsf{Sol}(z)$ *can be computed in time* $t \cdot \mathrm{poly}(\lambda)$.
– Sequentiality (security). *A time-lock puzzle* TLP *is* sequential *with* gap $\varepsilon < 1$ *if there exists a polynomial* $\underline{t}(\cdot)$*, such that for every polynomial* $t(\cdot) \geq \underline{t}(\cdot)$ *and every efficient parallel adversary* $\mathsf{A} = (\mathsf{A}_\lambda)_{\lambda \in \mathbb{N}}$ *with depth*$(\mathsf{A}_\lambda) \leq (t(\lambda))^\varepsilon$*, there exists a negligible function* $\delta \in \mathsf{negl}$*, such that for every* $\lambda \in \mathbb{N}$*, and every pair of solutions* $s_0, s_1 \in \{0,1\}^\lambda$*, we have that*

$$\Pr\left[\mathsf{A}(z) = b : \ b \leftarrow \{0,1\}, z \leftarrow \mathsf{PGen}(t(\lambda), s_b)\right] \leq \frac{1}{2} + \delta(\lambda).$$

Trapdoor Verifiable Delay Functions. We define the syntax and requirements of trapdoor verifiable delay function (TVDF) [62]. Recall that they are a more general notion than the (plain) VDFs from [15] (see Remark 1). A VDF is a function that requires specific number of (e.g., t) sequential steps to evaluate, yet produce a unique output that can be efficiently and publicly verified. Compared to plain VDFs, in TVDFs there is an additional trapdoor that allows to evaluate the function in strictly less time than performing t sequential steps.

Definition 8 (TVDF [62]). *A trapdoor verifiable delay function* TVDF *is a tuple of algorithms* $(\mathsf{Setup}, \mathsf{Eval}, \mathsf{TEval}, \mathsf{Ver})$*, with the following syntax:*

– $(\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)$. *On input a statistical security parameter* $\lambda \in \mathbb{N}$ *(in unary) and a time parameter* $t \in \mathbb{N}$*, the* set-up *algorithm outputs an evaluation key* ek*, a verification key* vk *and a trapdoor* td*. The keys* ek, vk *determine the domain* $\mathcal{X}$ *and range* $\mathcal{Y}$ *of the VDF function* $V : \mathcal{X} \to \mathcal{Y}$.
– $(y, \pi) \leftarrow \mathsf{Eval}(\mathsf{ek}, x)$. *On input* $x \in \mathcal{X}$*, the* evaluation *algorithm outputs* (y, π)*, where* π *is a proof that the output* $y \in \mathcal{Y}$ *has been correctly computed.*
– $(y, \pi) \leftarrow \mathsf{TEval}(\mathsf{td}, x)$. *On input a trapdoor* td *and input* $x \in \mathcal{X}$*, the* trapdoor-evaluation *algorithm outputs* (y, π) *as in the (plain) evaluation algorithm*
– $b := \mathsf{Ver}(\mathsf{vk}, x, y, \pi)$. *Given as input a tuple* (x, y, π) *consisting of an input, an output and a proof, the* proof-verification *algorithm outputs a bit* b *indicating accept* (1) *or reject* (0).

A TVDF must satisfy the following properties:

– Completeness. *Honestly-generated proofs must always accept, i.e., for any* $\lambda, t \in \mathbb{N}$ *and* $x \in \mathcal{X}$

$$\Pr[\mathsf{Ver}(\mathsf{vk}, x, \mathsf{Eval}(\mathsf{ek}, x)) = 1 : (\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)] = 1 \ .$$

– Consistency of trapdoor evaluations. *Honestly-generated trapdoor proofs must always accept, and honestly-generated trapdoor evaluations must always agree with honestly-generated evaluations, i.e., for any* $\lambda, t \in \mathbb{N}$ *and* $x \in \mathcal{X}$

$$\Pr\left[\begin{array}{rl} y = y' \ \wedge & (\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ \mathsf{Ver}(\mathsf{vk}, x, y, \pi) = 1 \ \wedge : & (y, \pi) \leftarrow \mathsf{Eval}(\mathsf{ek}, x) \\ \mathsf{Ver}(\mathsf{vk}, x, y', \pi') = 1 & (y', \pi') \leftarrow \mathsf{TEval}(\mathsf{td}, x) \end{array}\right] = 1 \ .$$

– Efficiency. $\mathsf{Setup}, \mathsf{TEval}$ *and* Ver *run in time* $\mathrm{poly}(\log(t), \lambda)$. Eval *computes the output* y *and its proof* π *in time* $(t + o(t)) \cdot \mathrm{poly}(\lambda)$.
– Soundness. *We define three different notions of soundness, in order of decreasing strength: perfect, adaptive computational and selective computational.*
  • Perfect *soundness requires accepting proofs for wrong evaluation* to not exist*. Formally, a TVDF is perfectly sound if*

$$\Pr\left[\begin{array}{r} \exists(x, y^*, \pi^*) \text{ s.t.} \\ \mathsf{Ver}(\mathsf{vk}, x, y^*, \pi^*) = 1 \wedge \\ y^* \neq y \end{array} : \begin{array}{l} (\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ (y, \pi) \leftarrow \mathsf{Eval}(\mathsf{ek}, x) \end{array}\right] = 0.$$

  • Adaptive computational *soundness requires it to be hard for an efficient adversary to come up with an accepting proof for a wrong evaluation. Formally, a TVDF is adaptively computationally sound if for all* $t \in \mathbb{N}$ *and efficient adversaries* $\mathsf{A} = (\mathsf{A}_\lambda)_{\lambda \in \mathbb{N}}$

$$\Pr\left[\begin{array}{r} \mathsf{Ver}(\mathsf{vk}, x^*, y^*_{\mathsf{A}}, \pi^*_{\mathsf{A}}) = 1 \wedge \\ y^*_{\mathsf{A}} \neq y^* \end{array} : \begin{array}{l} (\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ (x^*, y^*_{\mathsf{A}}, \pi^*_{\mathsf{A}}) \leftarrow \mathsf{A}_\lambda(\mathsf{ek}, \mathsf{vk}) \\ (y^*, \pi^*) \leftarrow \mathsf{Eval}(\mathsf{ek}, x^*) \end{array}\right] \in \mathsf{negl}(\lambda).$$

  • Selective computational *soundness weakens the above requirement to the case where the adversary submits the challenge input* in advance*. Formally, a TVDF is selectively computationally sound if for all* $t \in \mathbb{N}$ *and efficient two-stage adversaries* $\mathsf{A} = (\mathsf{A}_{0,\lambda}, \mathsf{A}_{1,\lambda})_{\lambda \in \mathbb{N}}$

$$\Pr\left[\begin{array}{r} \mathsf{Ver}(\mathsf{vk}, x^*, y^*_{\mathsf{A}}, \pi^*_{\mathsf{A}}) = 1 \wedge \\ y^*_{\mathsf{A}} \neq y^* \end{array} : \begin{array}{l} (\mathsf{st}, x^*) \leftarrow \mathsf{A}_{0,\lambda} \\ (\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ (y^*_{\mathsf{A}}, \pi^*_{\mathsf{A}}) \leftarrow \mathsf{A}_{1,\lambda}(\mathsf{st}, \mathsf{ek}, \mathsf{vk}) \\ (y^*, \pi^*) \leftarrow \mathsf{Eval}(\mathsf{ek}, x^*) \end{array}\right] \in \mathsf{negl}(\lambda).$$

– Sequentiality (security). *We consider two different notions of sequentiality: with and without preprocessing on public parameters.*
  • *A TVDF is* sequential *with* gap $\varepsilon < 1$ *if there exists a polynomial* $\underline{t}(\cdot)$*, such that for every polynomial* $t(\cdot) \geq \underline{t}(\cdot)$ *and every efficient two-stage adversary* $\mathsf{A} = (\mathsf{A}_{0,\lambda}, \mathsf{A}_{1,\lambda})_{\lambda \in \mathbb{N}}$*, where* $\mathsf{A}_{1,\lambda}$ *is parallel with depth*$(\mathsf{A}_{1,\lambda}) \leq (t(\lambda))^\varepsilon$*, it holds that*

$$\Pr\left[y^*_{\mathsf{A}} = y^* : \begin{array}{l} (\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ \mathsf{st} \leftarrow \mathsf{A}_{0,\lambda}(\mathsf{ek}, \mathsf{vk}) \\ x^* \leftarrow \mathcal{X} \\ y^*_{\mathsf{A}} \leftarrow \mathsf{A}_{1,\lambda}(\mathsf{st}, x^*) \\ (y^*, \pi^*) \leftarrow \mathsf{Eval}(\mathsf{ek}, x^*) \end{array}\right] \in \mathsf{negl}(\lambda).$$

  • *A TVDF is* sequential on fresh parameters *with* gap $\varepsilon < 1$ *if there exists a polynomial* $\underline{t}(\cdot)$*, such that for every polynomial* $t(\cdot) \geq \underline{t}(\cdot)$ *and every*

efficient adversary $\mathsf{A} = (\mathsf{A}_\lambda)_{\lambda\in\mathbb{N}}$*, where* A_λ *is parallel with depth*$(\mathsf{A}_\lambda) \leq (t(\lambda))^\varepsilon$*, it holds that*

$$\Pr\left[y^*_{\mathsf{A}} = y^* : \begin{array}{l}(\mathsf{ek},\mathsf{vk},\mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)\\ x^* \leftarrow \mathcal{X}\\ y^*_{\mathsf{A}} \leftarrow \mathsf{A}_\lambda(\mathsf{ek},\mathsf{vk},x^*)\\ (y^*,\pi^*) \leftarrow \mathsf{Eval}(\mathsf{ek},x^*)\end{array}\right] \in \mathsf{negl}(\lambda).$$

Remark 1 (Standard VDFs). Note that the standard "trapdoorless" definition of VDFs can be recovered from Definition 8 by ignoring td and TEval.

Additionally, we require a pseudorandomness property from [15], which requires the output of the VDF to appear pseudorandom to sequentiality adversaries.

Definition 9 (TVDFs with pseudorandom outputs). *Analogous to sequentiality, we consider two notions: with and without preprocessing.*

– *A TVDF* $(\mathsf{Setup},\mathsf{Eval},\mathsf{TEval},\mathsf{Ver})$ *has* pseudorandom *outputs with* gap $\varepsilon < 1$ *if there exists a polynomial* $\underline{t}(\cdot)$*, such that for every polynomial* $t(\cdot) \geq \underline{t}(\cdot)$ *and every efficient two-stage distinguisher* $\mathsf{A} = (\mathsf{A}_{0,\lambda},\mathsf{A}_{1,\lambda})_{\lambda\in\mathbb{N}}$*, where* $\mathsf{A}_{1,\lambda}$ *is parallel with depth*$(\mathsf{A}_{1,\lambda}) \leq (t(\lambda))^\varepsilon$*, it holds that*

$$\left|\Pr\left[\mathsf{A}_{1,\lambda}(\mathsf{st},x^*,y^*) = 0 : \begin{array}{l}(\mathsf{ek},\mathsf{vk},\mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)\\ \mathsf{st} \leftarrow \mathsf{A}_{0,\lambda}(\mathsf{ek},\mathsf{vk})\\ x^* \leftarrow \mathcal{X}\\ (y^*,\pi^*) \leftarrow \mathsf{Eval}(\mathsf{ek},x^*)\end{array}\right] - \Pr\left[\mathsf{A}_{1,\lambda}(\mathsf{st},x^*,y^*) = 0 : \begin{array}{l}(\mathsf{ek},\mathsf{vk},\mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)\\ \mathsf{st} \leftarrow \mathsf{A}_{0,\lambda}(\mathsf{ek},\mathsf{vk})\\ x^* \leftarrow \mathcal{X}\\ y^* \leftarrow \mathcal{Y}\end{array}\right]\right| \in \mathsf{negl}(\lambda) \quad (2)$$

– *A TVDF* $(\mathsf{Setup},\mathsf{Eval},\mathsf{TEval},\mathsf{Ver})$ *has* pseudorandom *outputs* without preprocessing *with* gap $\varepsilon < 1$ *if there exists a polynomial* $\underline{t}(\cdot)$*, such that for every polynomial* $t(\cdot) \geq \underline{t}(\cdot)$ *and every efficient distinguisher* $\mathsf{A} = (\mathsf{A}_\lambda)_{\lambda\in\mathbb{N}}$*, where* A_λ *is parallel with depth*$(\mathsf{A}_\lambda) \leq (t(\lambda))^\varepsilon$*, it holds that*

$$\left|\Pr\left[\mathsf{A}_\lambda(\mathsf{ek},\mathsf{vk},x^*,y^*) = 0 : \begin{array}{l}(\mathsf{ek},\mathsf{vk},\mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)\\ x^* \leftarrow \mathcal{X}\\ (y^*,\pi^*) \leftarrow \mathsf{Eval}(\mathsf{ek},x^*)\end{array}\right] - \Pr\left[\mathsf{A}_\lambda(\mathsf{ek},\mathsf{vk},x^*,y^*) = 0 : \begin{array}{l}(\mathsf{ek},\mathsf{vk},\mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)\\ x^* \leftarrow \mathcal{X}\\ y^* \leftarrow \mathcal{Y}\end{array}\right]\right| \in \mathsf{negl}(\lambda) \quad (3)$$

3 Trapdoor VDF from TLP

In this section we present VDF_1, our basic construction of TVDF from TLP. The construction is formally described in Figure 1. Note that the domain of VDF_1

for parameters (λ, t) is $\mathcal{X}_\lambda := \{0,1\}^\lambda$, while the range is $\mathcal{Y}_\lambda := \{0,1\}^{m(\lambda)}$, the range of the PPRF.[16] The function (assuming iO is perfectly correct) itself is defined as $V(x) := \mathsf{FEval}(k_1, x\|t)$. A formal proof that VDF_1 is a TVDF follows in Theorem 1.

Given an ensemble of IOWF f, a PPRF family $\mathsf{PPRF} = (\mathsf{KGen}, \mathsf{FEval}, \mathsf{Puncture})$, a TLP $\mathsf{TLP} = (\mathsf{PGen}, \mathsf{Sol})$ and an IO iO, all defined as in Theorem 1, the construction of our TVDF $\mathsf{VDF}_1 = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{TEval}, \mathsf{Ver})$ is described below.

$(\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1,k_2]}(\cdot))$ where

   $(z, \pi) := \mathsf{Sample}_{[t,k_1,k_2]}(x)$

   (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1, t\|x)$
   (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
   (c) Compute the "proof" $\pi := f_\lambda(s)$
   (d) Output (z, π)
3. Set $\mathsf{vk} := \mathsf{ek}$, $\mathsf{td} := k_1$ and output $(\mathsf{ek}, \mathsf{vk}, \mathsf{td})$

$(s, \perp) := \mathsf{Eval}(\mathsf{ek}, x)$

1. Use ek to generate a puzzle: $(z, \pi) := \mathsf{ek}(x)$
2. Solve the TLP to compute the output of the VDF: $s := \mathsf{Sol}(z)$
3. Output $(s, \perp)$, where $\perp$ denotes an empty proof

$(s, \perp) := \mathsf{TEval}(\mathsf{td}, x)$

1. Output $(s, \perp)$, where $s := \mathsf{FEval}(\mathsf{td}, t\|x)$

$b := \mathsf{Ver}(\mathsf{vk}, x, s, \perp)$

1. Use vk to re-generate the puzzle: $(z, \pi) := \mathsf{vk}(x)$
2. Accept (i.e., output $b := 1$) if and only if the "proof" verifies, i.e., $f_\lambda(s) = \pi$.

Fig. 1. VDF_1: Basic construction of TVDF from TLP.

Theorem 1. *For a polynomial $m(\cdot)$, let $\mathcal{F} := \left(F_\lambda : \mathcal{K}_\lambda \times \{0,1\}^{2\lambda} \to \{0,1\}^{m(\lambda)}\right)_{\lambda \in \mathbb{N}}$ be a PPRF family defined using $\mathsf{PPRF} = (\mathsf{KGen}, \mathsf{FEval}, \mathsf{Puncture})$ (Definition 5).*

[16] Here we assume that t is at most 2^λ and thus $|t| \leq \lambda$. Thus, whenever the input to PPRF is shorter than 2λ bits, we assume that it is appropriately padded.

Let $f = \{f_\lambda : \{0,1\}^{m(\lambda)} \to \{0,1\}^{2m(\lambda)}\}_{\lambda\in\mathbb{N}}$ *be a family of injective one-way functions. Moreover, let* $\mathsf{TLP} = (\mathsf{PGen}, \mathsf{Sol})$ *be any time-lock puzzle (Definition 7) and* iO *be any indistinguishability obfuscator (Definition 2). Then the construction* $\mathsf{VDF}_1 = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{TEval}, \mathsf{Ver})$ *described in Figure 1 is a TVDF. In particular,*

- VDF_1 *is* perfectly *sound,*
- *if* Sol *runs in time* $t \cdot p_1(\lambda)$, Sample *runs in time* $p_2(\lambda)$*, and IO has an efficiency overhead* $p_3(\lambda)$ *for some polynomials* $p_1(\cdot)$ $p_2(\cdot)$ *and* $p_3(\cdot)$ *then* Eval *runs in time* $(t + \log^{O(1)}(t)) \cdot p(\lambda)$*, where the polynomial* $p(\cdot)$ *is determined by* $p_i(\cdot)$*s.*
- *if* TLP *is sequential with a gap* ϵ*,* iO *and* PPRF *are secure, and* f *is one-way then* VDF_1 *is sequential without preprocessing with gap* $O(\epsilon)$*,*
- *if* TLP *is sequential with a gap* ϵ*,* iO *and* PPRF *are secure, and* f *is one-way then* VDF_1 *is pseudorandom without preprocessing with gap* $O(\epsilon)$*.*

Remark 2. While in Theorem 1 we consider iO that is secure against PPT adversaries, we note that we can relax this requirement and consider only fine-grained iO. Concretely, since in the model without preprocessing the VDF adversary is purely of bounded depth, it suffices to also consider iO that is secure against the same bounded depth adversaries.

Proof. The completeness of the construction follows from the correctness of the underlying primitives (i.e., IO and TLP). The efficiency of the construction is straightforward to establish. In the rest of the proof, we focus on soundness, sequentiality and pseudorandomness, which we prove in the following claims.

Claim. VDF_1 is *perfectly* sound.

Proof. Note that in our construction the VDF proof output by the evaluation is empty, and that the verifier simply checks whether $f_\lambda(s) = \pi$, where π is generated using (honestly-generated) ek. By perfect correctness of iO and TLP, for an adversary's output $(x, s^*, \perp)$, such that $s^* \neq s := \mathsf{FEval}(k_1, t\|x)$, to be accepted it must be that $f_\lambda(s^*) = \pi = f_\lambda(s)$. However, this contradicts f's injectivity. □

Claim. If TLP is sequential with gap ϵ, iO is perfectly correct, iO and PPRF are secure, and f is one-way then VDF_1 is also sequential without preprocessing with gap $O(\epsilon)$.

Proof. We proceed by a hybrid argument consisting of six hybrids $\mathbf{Hybrid}_0, \cdots, \mathbf{Hybrid}_5$, which we describe below. Each hybrid is parametrised by the VDF adversary A against sequentiality without preprocessing (see Definition 8), which we drop to avoid clutter. The difference from the previous hybrid is highlighted in red.

- $\mathbf{Hybrid}_0$ is the distribution of the "real" sequentiality experiment. Hence it is generated according to the protocol description in Figure 1 and the experiment outputs 1 if and only if the adversary breaks sequentiality of VDF_1. Note that assuming perfect correctness of iO and PPRF, the winning condition simplifies to "$s^*_{\mathsf{A}} = \mathsf{FEval}(k_1, t\|x^*)$".

$\mathbf{Hybrid}_0(1^\lambda, t)$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1,k_2]}(\cdot))$ and set $\mathsf{vk} := \mathsf{ek}$, where

   $\underline{(z, \pi) := \mathsf{Sample}_{[t,k_1,k_2]}(x)}$

   (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1, t\|x)$
   (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
   (c) Compute the "proof" $\pi := f_\lambda(s)$
   (d) Output (z, π)
3. Sample a random challenge $x^* \leftarrow \mathcal{X}_\lambda$
4. Run A on $(\mathsf{ek}, \mathsf{vk}, x^*)$ to obtain s^*_A
5. Output 1 if and only if $s^*_\mathsf{A} = \mathsf{FEval}(k_1, t\|x^*)$

– In $\mathbf{Hybrid}_1$, we puncture the key k_1 at the challenge $\{t\|x^*\}$. The resulting distribution is indistinguishable from $\mathbf{Hybrid}_0$ thanks to preservation of PPRF functionality under puncturing and IO security.

$\mathbf{Hybrid}_1(1^\lambda, t)$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample a random challenge $x^* \leftarrow \mathcal{X}_\lambda$, and a punctured key $k_1^* \leftarrow \mathsf{Puncture}(k_1, \{t\|x^*\})$
3. Pre-compute the *challenge* solution-puzzle pair with a proof:
   (a) Generate a pseudorandom TLP solution $s^* := \mathsf{FEval}(k_1, t\|x^*)$
   (b) Use s^* and *pseudorandom* coins to generate a TLP puzzle $z^* := \mathsf{PGen}(s^*, t; \mathsf{FEval}(k_2, t\|x^*))$
   (c) Compute the challenge "proof" $\pi^* := f_\lambda(s^*)$
4. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1^*,k_2,x^*,z^*,\pi^*]}(\cdot))$ and set $\mathsf{vk} := \mathsf{ek}$, where

   $(z, \pi) := \mathsf{Sample}_{[t,k_1^*,k_2,x^*,z^*,\pi^*]}(x)$

   - If $x = x^*$ then output (z^*, π^*)
   - Else
     (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1^*, t\|x)$
     (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
     (c) Compute the "proof" $\pi := f_\lambda(s)$
     (d) Output (z, π)
5. Run A on $(\mathsf{ek}, \mathsf{vk}, x^*)$ to obtain s^*_A
6. Output 1 if and only if $s^*_\mathsf{A} = s^*$

– $\mathbf{Hybrid}_2$ is similar to $\mathbf{Hybrid}_1$ except that the challenge solution s^* is sampled at random. The resulting distribution is indistinguishable from $\mathbf{Hybrid}_0$ thanks to pseudorandomness of PPRF at punctured points.

$\mathbf{Hybrid}_2(1^\lambda, t)$

⋮

3. Pre-compute the *challenge* solution-puzzle pair with a proof:
   (a) Sample a *random* TLP solution $s^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Generate the TLP puzzle $z^* := \mathsf{PGen}(s^*, t; \mathsf{FEval}(k_2, t\|x^*))$ from s^* using *pseudorandom* coins
   (c) Compute the "proof" $\pi^* := f_\lambda(s^*)$

⋮

6. Output 1 if and only if $s^*_\mathsf{A} = s^*$

– In $\mathbf{Hybrid}_3$, we puncture the key k_2 at $\{t\|x^*\}$. Looking ahead, the purpose of this step is to undo the "derandomisation" carried out in Setup for generating

the TLP puzzle. As for $\mathbf{Hybrid}_1$, this change is indistinguishable thanks to preservation of PPRF functionality under puncturing and IO security.

$\mathbf{Hybrid}_3(1^\lambda, t)$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample a random challenge $x^* \leftarrow \mathcal{X}_\lambda$, and a punctured key $k_1^* \leftarrow \mathsf{Puncture}(k_1, \{t\|x^*\})$
3. Pre-compute the *challenge* solution-puzzle pair with a proof:
   (a) Sample a *random* TLP solution $s^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Sample a punctured key $k_2^* \leftarrow \mathsf{Puncture}(k_2, \{t\|x^*\})$
   (c) Use s^* and *pseudorandom* coins to generate the TLP puzzle $z^* := \mathsf{PGen}(s^*, t; \mathsf{FEval}(k_2, t\|x^*))$
   (d) Compute the "proof" $\pi^* := f_\lambda(s^*)$
4. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1^*,k_2^*,x^*,z^*,\pi^*]}(\cdot))$ and set $\mathsf{vk} := \mathsf{ek}$, where

   $(z, \pi) := \mathsf{Sample}_{[t,k_1^*,k_2^*,x^*,z^*,\pi^*]}(x)$

   - If $x = x^*$ then output (z^*, π^*)
   - Else
     (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1^*, t\|x)$
     (b) Use s and *pseudorandom* coins to generate the TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2^*, t\|x))$
     (c) Compute the "proof" $\pi := f_\lambda(s)$
     (d) Output (z, π)
5. Run A on $(\mathsf{ek}, \mathsf{vk}, x^*)$ to obtain s_A^*
6. Output 1 if and only if $s_\mathsf{A}^* = s^*$

– In $\mathbf{Hybrid}_4$, we switch the random coins used for generating the challenge TLP puzzle from pseudorandom to truly random. This is necessary to invoke sequentiality of TLP in the next step. As for $\mathbf{Hybrid}_2$, this change is indistinguishable thanks to pseudorandomness of PPRF at punctured points.

Hybrid$_4(1^\lambda, t)$

$\vdots$

3. Pre-compute the *challenge* solution-puzzle pair with a proof:
   (a) Sample a *random* TLP solution $s^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Sample a punctured key $k_2^* \leftarrow \mathsf{Puncture}(k_2, \{t \| x^*\})$
   (c) Sample random coins $r^* \leftarrow \{0,1\}^{\mathrm{poly}(\lambda)}$
   (d) Use s^* and *random* coins to generate the TLP puzzle $z^* := \mathsf{PGen}(s^*, t; r^*)$
   (e) Compute the "proof" $\pi^* := f_\lambda(s^*)$

$\vdots$

6. Output 1 if and only if $s^*_{\mathsf{A}} = s^*$

– Finally, in **Hybrid**$_5$, we decouple the proof from the puzzle: a *second* solution s_2^* is used to generate the challenge puzzle (but the adversary wins if it still outputs s_1^*). However, this change is indistinguishable to *bounded-depth* adversaries thanks to TLP sequentiality without preprocessing, and thus the probabilities that a bounded-depth adversary outputs s_1^* in **Hybrid**$_4$ and **Hybrid**$_5$ are negligibly close.

Hybrid$_5(1^\lambda, t)$

$\vdots$

3. Pre-compute the *challenge* solution-puzzle pair with a proof:
   (a) Sample a *random* TLP solution $s_1^*, s_2^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Sample a punctured key $k_2^* \leftarrow \mathsf{Puncture}(k_2, \{t \| x^*\})$
   (c) Sample random coins $r^* \leftarrow \{0,1\}^{\mathrm{poly}(\lambda)}$
   (d) Use s_2^* and *random* coins to generate the TLP puzzle $z_2^* := \mathsf{PGen}(s_2^*, t; r^*)$
   (e) Compute the "proof" $\pi^* := f_\lambda(s_1^*)$

$\vdots$

6. Output 1 if and only if $s^*_{\mathsf{A}} = s_1^*$

The probability that experiment **Hybrid**$_5(1^\lambda, t)$ outputs 1 can be bounded by the one-wayness of f since the only the proof π^* has any information about s_1^*. Since the depth of all the reductions depends on t only by a $\log(t)$ factor, the gap of VDF is sequential with $O(\epsilon)$ gap.

□

Claim. If TLP is sequential with a gap ϵ, iO and PPRF are secure, and f is one-way then VDF_1 is also pseudorandom without preprocessing with gap $O(\epsilon)$.

Proof. This follows from the proof of sequentiality above by observing that the distribution of s_2^* in $\mathbf{Hybrid}_5$ is random. □

This completes the proof of Theorem 1. □

Remark 3. We can potentially consider advanced equivocation techniques to allow to embed the TLP into the input x^* instead of the public parameters, and hence, only embed the TLP challenge at a later point during the security game. However, unfortunately, we do not see how to instantiate this idea, and we believe it will require new ideas. The main difficulty here seems to be that puncturing techniques only allow to change the output of the obfuscated circuit at certain *predetermined* input points. Concretely, we would require some input x^*, that to some extent is set already in the public parameters, to later be mapped to some z^* which is unknown at the time of public parameter generation but can be extracted from x^*. Hence, we should only fix some partial information on x^* in the public parameters, such that later we can embed the TLP z^*. On the other hand, if the obfuscated circuit maps all inputs x with the same partial information as x^* to a new TLP derived from x in a different way than the original circuit, this would mean that we need to reprogram the circuit on many inputs at the same time.

Corollary 1 (VDF_1 from any non-parallelizing language, iO and OWF). *Assuming the existence of non-parallelizing languages with worst-case hardness, polynomially-secure iO for circuits and polynomially-hard one-way functions, there exist TVDFs without preprocessing.*

Proof. This is a consequence of Theorem 1, taking into account the prior work in [11, 13]:

1. [11] established that TLPs exist assuming worst-case hardness of non-parallelising languages and polynomially-secure iO for circuits.
2. [13] showed that it is possible to construct IOWF from polynomially-secure iO and polynomially-hard OWF.

□

4 Efficient-Verifier Trapdoor VDF from TLP

In this section we present VDF_2, our second construction of TVDF from TLP. The main advantage of VDF_2 over VDF_1 from Figure 1 is that the verifier – since it doesn't have to invoke an obfuscated program – is more efficient. To this end, we additionally rely on prefix-constrained signatures (PCS, see Definition 6). The assumptions relied upon are the same as in the basic construction since prefix-constrained signatures can be constructed based on OWF (see Section 4.1 below). However, the soundness achieved by the construction is only selective. The construction is formally described in Figure 2, and then we prove in Theorem 2 that it is a TVDF. Note that the VDF function is exactly as in the basic construction.

Given a family of OWP f, a PPRF $\mathsf{PPRF} = (\mathsf{KGen}, \mathsf{FEval}, \mathsf{Puncture})$, a PCS $\mathsf{PCS} := (\mathsf{Gen}, \mathsf{CGen}, \mathsf{Sign}, \mathsf{CSign}, \mathsf{Ver})$, a TLP $\mathsf{TLP} = (\mathsf{PGen}, \mathsf{Sol})$ and an IO iO, all defined as in Theorem 2, the construction of our TVDF $\mathsf{VDF}_2 = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{TEval}, \mathsf{Ver})$ is described below.

$\underline{(\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t)}$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample a PCS key-pair $(\mathsf{pk}, \mathsf{msk}) \leftarrow \mathsf{Gen}(1^\lambda)$
3. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1,k_2,\mathsf{msk}]}(\cdot))$ where

   $\underline{(z, \pi, \sigma) := \mathsf{Sample}_{[t,k_1,k_2,\mathsf{msk}]}(x)}$

   (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1, t\|x)$
   (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
   (c) Compute the proof $\pi := f_\lambda(s)$ and signature $\sigma := \mathsf{Sign}(\mathsf{msk}, x\|\pi\|z)$
   (d) Output (z, π, σ)
4. Set $\mathsf{vk} := (\mathsf{ek}, \mathsf{pk})$, $\mathsf{td} := (\mathsf{ek}, k_1)$ and output $(\mathsf{ek}, \mathsf{vk}, \mathsf{td})$

$\underline{(s, (z, \pi, \sigma)) := \mathsf{Eval}(\mathsf{ek}, x)}$

1. Use ek to generate a puzzle: $(z, \pi, \sigma) := \mathsf{ek}(x)$
2. Solve the TLP to compute the output of the VDF: $s := \mathsf{Sol}(z)$
3. Output $(s, (z, \pi, \sigma))$

$\underline{(s, (z, \pi, \sigma)) := \mathsf{TEval}(\mathsf{td}, x)}$

1. Use ek to generate a puzzle: $(z, \pi, \sigma) := \mathsf{ek}(x)$
2. Use k_1 to generate the solution $s := \mathsf{FEval}(k_1, t\|x)$
3. Output $(s, (z, \pi, \sigma))$

$\underline{b := \mathsf{Ver}(\mathsf{vk}, x, s, (z, \pi, \sigma))}$

1. Accept (i.e., output $b := 1$) if and only if both proof and signature verify, i.e.,
   (a) $f_\lambda(s) = \pi$ and
   (b) $\mathsf{Ver}(\mathsf{pk}, \sigma, x\|\pi\|z) = 1$

Fig. 2. VDF_2, an efficient-verifier TVDF from TLP.

Theorem 2. *For a polynomial $m(\cdot)$, let $\mathcal{F} := \left(F_\lambda : \mathcal{K}_\lambda \times \{0,1\}^{2\lambda} \to \{0,1\}^{m(\lambda)}\right)_{\lambda\in\mathbb{N}}$ be a PPRF family defined using $\mathsf{PPRF} = (\mathsf{KGen}, \mathsf{FEval}, \mathsf{Puncture})$ (Definition 5). Let $f = \{f_\lambda : \{0,1\}^{m(\lambda)} \to \{0,1\}^{2m(\lambda)}\}_{\lambda\in\mathbb{N}}$ be an ensemble of injective one-way functions. Let $\mathsf{TLP} = (\mathsf{Gen}, \mathsf{Sol})$ be any time-lock puzzle with puzzles of length $w(\cdot)$ (Definition 7). Moreover, let $\mathsf{PCS} := (\mathsf{KGen}, \mathsf{CGen}, \mathsf{Sign}, \mathsf{CSign}, \mathsf{Ver})$ be any prefix-constrained signature with message-space defined by $\ell(\cdot) := \lambda+2m(\cdot)+w(\cdot)$ (Definition 6) and iO be any indistinguishability obfuscator (Definition 2). Then the construction $\mathsf{VDF}_2 = (\mathsf{Setup}, \mathsf{Eval}, \mathsf{TEval}, \mathsf{Ver})$ described in Figure 2 is a TVDF. In particular,*

- *if* PCS *is selectively one-time constrained unforgeable then* VDF_2 *is* selectively computationally *sound,*
- *if* Sol *runs in time $t \cdot p_1(\lambda)$,* Sample *runs in time $p_2(\lambda)$, and IO has an efficiency overhead $p_3(\lambda)$ for some polynomials $p_1(\cdot)$ $p_2(\cdot)$ and $p_3(\cdot)$ then* Eval *runs in time $(t + \log^{O(1)}(t)) \cdot p(\lambda)$, where the polynomial $p(\cdot)$ is determined by $p_i(\cdot)$s.*
- *if* TLP *is sequential with a gap ϵ,* iO *and* PPRF *are secure, and f is one-way then* VDF_2 *is sequential without preprocessing with gap $O(\epsilon)$,*
- *if* TLP *is sequential with a gap ϵ,* iO *and* PPRF *are secure, and f is one-way then* VDF_2 *is pseudorandom without preprocessing with gap $O(\epsilon)$.*

Proof. The correctness of the construction follows from the correctness of the underlying primitives (i.e., IO, TLP and PCS). The efficiency of the construction is straightforward to establish. In the rest of the proof, we focus on soundness, sequentiality and pseudorandomness, which we prove in the following claims.

Claim. If PCS is selectively one-time constrained unforgeable then VDF_2 is *selectively computationally* sound.

Proof. We proceed by a hybrid argument that consists of two hybrids $\mathbf{Hybrid}_0$ and $\mathbf{Hybrid}_1$, which we describe below.

- $\mathbf{Hybrid}_0$ is the distribution of the "real" soundness experiment. Hence it is generated according to the protocol description in Figure 2 and the experiment outputs 1 if and only if the adversary breaks soundness of VDF_2.

- In $\mathbf{Hybrid}_1$, we precompute and hardcode the output of Sample on input x^* and switch the hardwired key from msk to a constrained signing key sk^* that is obtained by constraining msk at the prefix x^*. Hybrids $\mathbf{Hybrid}_0$ and $\mathbf{Hybrid}_1$ are indistinguishable by iO security and consistency of PCS on constrained points.

We claim that breaking soundness of VDF_2 in $\mathbf{Hybrid}_1$ is now tantamount to breaking selective unforgeability of PCS. To see this note that msk is used only to generate sk^* and the signature σ^* on message (x^*, π^*, z^*). In the reduction the reduction algorithm can simply invoke the constrained key generation on the string x^* to receive sk^* and query the signing oracle to receive σ^*. If an adversary

$\mathbf{Hybrid}_0(1^\lambda, t)$

1. Run $\mathsf{A}_{0,\lambda}$ to obtain $x^* \in \{0,1\}^\lambda$ and state st.
2. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
3. Sample a PCS key-pair $(\mathsf{pk}, \mathsf{msk}) \leftarrow \mathsf{Gen}(1^\lambda)$
4. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1,k_2,\mathsf{msk}]}(\cdot))$ and set $\mathsf{vk} := (\mathsf{ek}, \mathsf{pk})$, where

   $(z, \pi, \sigma) := \mathsf{Sample}_{[t,k_1,k_2,\mathsf{msk}]}(x)$

   (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1, t\|x)$
   (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
   (c) Compute the proof $\pi := f_\lambda(s)$ and signature $\sigma := \mathsf{Sign}(\mathsf{msk}, x\|\pi\|z)$
   (d) Output (z, π, σ)
5. Run $\mathsf{A}_{1,\lambda}$ on $(\mathsf{st}, \mathsf{ek}, \mathsf{vk})$ to obtain $(s^*_\mathsf{A}, (z^*_\mathsf{A}, \pi^*_\mathsf{A}, \sigma^*_\mathsf{A}))$.
6. Output 1 if and only if
   - $s^*_\mathsf{A} \neq \mathsf{PPRF.Eval}(k_0, x^*)$ and
   - $f_\lambda(s^*_\mathsf{A}) = \pi^*_\mathsf{A}$ and $\mathsf{Ver}(\mathsf{pk}, \sigma^*_\mathsf{A}, x^*\|\pi^*_\mathsf{A}\|z^*_\mathsf{A}) = 1$.

now breaks soundness of VDF_2, this means it output $(s^*_\mathsf{A}, (z^*_\mathsf{A}, \pi^*_\mathsf{A}, \sigma^*_\mathsf{A}))$ with $s^*_\mathsf{A} \neq \mathsf{PPRF.Eval}(k_0, x^*)$ such that $f_\lambda(s^*_\mathsf{A}) = \pi^*_\mathsf{A}$ and $\mathsf{Ver}(\mathsf{pk}, \sigma^*_\mathsf{A}, x^*\|\pi^*_\mathsf{A}\|z^*_\mathsf{A}) = 1$. Since f_λ is injective, we have $\pi^*_\mathsf{A} \neq \pi^*$, hence $(x^*\|\pi^*_\mathsf{A}\|z^*_\mathsf{A}, \sigma^*_\mathsf{A})$ gives a valid forgery. □

Claim. If TLP is sequential with gap ϵ, iO and PPRF are secure, and f is one-way then VDF_2 is also sequential without preprocessing with gap $O(\epsilon)$.

The proof of this claim is similar to that of sequentiality of the basic construction, except the hybrids are adapted to the construction VDF_2 in Figure 2. Therefore the proof is deferred to Appendix A.

Claim. If TLP is sequential with a gap ϵ, iO and PPRF are secure, and f is one-way then VDF_2 is also pseudorandom without preprocessing with gap $O(\epsilon)$.

Proof. This follows from the proof of sequentiality above by observing that the distribution of s^*_2 in $\mathbf{Hybrid}_5$ is random. □

This completes the proof of Theorem 2. □

4.1 Prefix-constrained signatures from OWF

We argue that the *puncturable* signature scheme from [21], built from PPRF and one-time signatures (OTS), already gives rise to a prefix-constrained signature scheme as per Definition 6. Their construction follows the paradigm of

Hybrid$_1(1^\lambda, t)$

1. Run $\mathsf{A}_{0,\lambda}$ to obtain $x^* \in \{0,1\}^{n(\lambda)}$ and state st.
2. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
3. Sample a PCS key-pair $(\mathsf{pk}, \mathsf{msk}) \leftarrow \mathsf{Gen}(1^\lambda)$
4. Pre-compute the solution $s^* := \mathsf{FEval}(k_1, t\|x^*)$, TLP puzzle $z^* := \mathsf{PGen}(s^*, t; \mathsf{FEval}(k_2, t\|x^*))$, proof $\pi^* := f_\lambda(s^*)$ and signature $\sigma^* := \mathsf{Sign}(\mathsf{msk}, x^*\|\pi^*\|z^*)$ on x^*
5. Generate a constrained key $\mathsf{sk}^* := \mathsf{CGen}(\mathsf{msk}, x^*)$
6. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1,k_2,\mathsf{sk}^*]}(\cdot))$ and set $\mathsf{vk} := (\mathsf{ek}, \mathsf{pk})$, where

   $(z, \pi, \sigma) := \mathsf{Sample}_{[t,k_1,k_2,\mathsf{sk}^*,\sigma^*]}(x)$

   (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1, t\|x)$
   (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
   (c) Compute the proof $\pi := f_\lambda(s)$, and signature σ as $\mathsf{Sign}(\mathsf{sk}^*, x\|\pi\|z)$ if $x \neq x^*$ and set $\sigma := \sigma^*$ if $x = x^*$
   (d) Output (z, π, σ)
7. Run $\mathsf{A}_{1,\lambda}$ on $(\mathsf{st}, \mathsf{ek}, \mathsf{vk})$ to obtain $(s^*_\mathsf{A}, (z^*_\mathsf{A}, \pi^*_\mathsf{A}, \sigma^*_\mathsf{A}))$.
8. Output 1 if and only if

   - $s^*_\mathsf{A} \neq \mathsf{PPRF.Eval}(k_0, x^*)$ and
   - $f_\lambda(s^*_\mathsf{A}) = \pi^*_\mathsf{A}$ and $\mathsf{Ver}(\mathsf{pk}, \sigma^*_\mathsf{A}, x^*\|\pi^*_\mathsf{A}\|z^*_\mathsf{A}) = 1$.

constructing (many-time) signatures from OTS using a tree of pseudorandomly generated one-time signatures. The seeds used to derive the one-time signature key pairs are derived using a PPRF. We note that OTS as well as PPRF can be constructed from OWF, namely by the one-time signature scheme of Lamport [44] and the PPRF construction of Goldreich, Goldwasser and Micali [35]. However, the construction of [21] cannot directly be instantiated with the GGM PPRF due to a mismatch of input domain as explained next.

In fact, for the security of the puncturable signature scheme one requires a PPRF with *variable-length* input domain $\mathcal{X} = \{0,1\}^{\leq \ell}$ opposed to fixed-length as in the GGM construction. Unfortunately, the basic GGM PPRF is *not* secure when punctured on a prefix. A generalisation of GGM, however, easily solves this issue: For each $i \leq \ell$ one can sample a separate key $k_i \leftarrow \mathsf{KGen}_{\mathsf{GGM}}(1^\lambda)$ according to the GGM construction and then define a PPRF with input space $\mathcal{X} = \{0,1\}^{\leq \ell}$ as $\mathsf{Eval}((k_i)_{i \leq \ell}, x) := \mathsf{Eval}_{\mathsf{GGM}}(k_{|m|}, m)$. It is easy to see that this construction can be proven secure based on OWF in the same way as the GGM construction itself.

Another useful property of the GGM construction is that it gives rise to a *constrained* PRF for the constraint classes of prefixes and "dual" prefixes, as required for our application. Using the above construction of a variable-length PPRF based on the GGM PPRF, one can therefore obtain a constrained PRF based on OWF with input domain $\mathcal{X} = \{0,1\}^{\leq \ell}$ supporting constraints of the form

$$S_{x^*} := \{0,1\}^{\leq \ell} \setminus \left(\{x\}_{x \prec x^*} \cup x^* \| \{0,1\}^{\leq \ell - |x^*|} \right), \tag{4}$$

i.e., the set consisting of all input strings *except for* prefixes and extensions of some $x^* \in \{0,1\}^{\leq \ell}$.

Our construction of PCS is the same tree-based construction as the puncturable signature scheme [21], but based on constrained PRF for the above more general constraint class, see Fig. 3 for how to generate constrained keys and use them for signing. To prove that the construction is indeed a selectively one-time prefix-constrained signature scheme, one can generalise [21, Theorem 1] to the different constraint class.

Lemma 1. *For a polynomial ℓ_1, let $\mathsf{OTS} = (\mathsf{Gen}_1, \mathsf{Sign}_1, \mathsf{Ver}_1)$ be a one-time signature scheme with public-key space $\{0,1\}^{\ell_1}$, message space $\{0,1\}^{2\ell_1 + \log \ell_1 + 1}$ and randomness space $\mathcal{R}_1$ (Definition 1). Let $\mathsf{CPRF} = (\mathsf{KGen}, \mathsf{FEval}, \mathsf{Constr})$ be a constrained PRF for the constraint class*

$$\mathcal{S} = \left\{ S_{x^*} \mid x^* \in \{0,1\}^{\leq \ell} \right\},$$

where S_{x^*} as in Equation (4), with $\mathcal{X} := \{0,1\}^{\leq \ell}$ with $\ell := 2\ell_1$ and $\mathcal{Y} = \mathcal{R}_1$ (Definition 4). Then the construction $\mathsf{PCS} = (\mathsf{Gen}, \mathsf{CGen}, \mathsf{Sign}, \mathsf{CSign}, \mathsf{Ver})$ described in Fig. 3 is a PCS for message space $\mathcal{M} := \{0,1\}^\ell$ (Definition 6).

Proof (Sketch). Correctness of PCS w.r.t. the master key as well as constrained signing keys follows from correctness of OTS and CPRF. For selective one-time constrained unforgeability, we follow a hybrid proof. The first hybrid $\mathbf{Hybrid}_0$ is

Given a one-time signature scheme $\mathsf{OTS} = (\mathsf{Gen}_1, \mathsf{Sign}_1, \mathsf{Ver}_1)$ with public-key space $\{0,1\}^{\ell_1}$, message space $\{0,1\}^{2\ell_1+\log \ell_1+1}$ and randomness space $\mathcal{R}_1$, and a constrained PRF $\mathsf{CPRF} = (\mathsf{KGen}, \mathsf{FEval}, \mathsf{Constr})$ with $\mathcal{X} := \{0,1\}^{\leq \ell}$ with $\ell = 2\ell_1$ and $\mathcal{Y} = \mathcal{R}_1$, a PCS $\mathsf{PCS} = (\mathsf{Gen}, \mathsf{CGen}, \mathsf{Sign}, \mathsf{CSign}, \mathsf{Ver})$ with message space $\{0,1\}^{\ell}$ is described below.

- $\underline{(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell)}$
  1. Sample a CPRF key $k \leftarrow \mathsf{KGen}(1^\lambda)$
  2. Compute $r_\epsilon := \mathsf{FEval}(k, \epsilon)$ and $(\mathsf{sk}_\epsilon, \mathsf{pk}_\epsilon) := \mathsf{Gen}_1(1^\lambda; r_\epsilon)$
  3. Output the PCS key-pair $(\mathsf{msk} := (k, \ell), \mathsf{pk} := (\mathsf{pk}_\epsilon, \ell))$

- $\underline{\mathsf{sk}^* \leftarrow \mathsf{CGen}(\mathsf{msk}, m^*)}$
  1. Sample a constrained key $k^* \leftarrow \mathsf{Constr}(k, S_{m^*})$ for S_{m^*} as in Eq.4
  2. For each $u \prec m^*$:
     (a) Let $r_u := \mathsf{FEval}(k, u)$ and $(\mathsf{sk}_u, \mathsf{pk}_u) := \mathsf{Gen}_1(1^\lambda; r_u)$
     (b) Compute $\sigma_u := \mathsf{Sign}_1(\mathsf{sk}_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))$
  3. Output $sk^* := (k^*, (\sigma_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))_{u \prec m^*})$

- $\underline{\sigma := \mathsf{Sign}(\mathsf{msk}, m)}$
  1. For each $u \preceq m$, let $r_u := \mathsf{FEval}(k, u)$ and $(\mathsf{sk}_u, \mathsf{pk}_u) := \mathsf{Gen}_1(1^\lambda; r_u)$
  2. For each $u \prec m$, compute $\sigma_u := \mathsf{Sign}_1(\mathsf{sk}_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))$
  3. Output the tuple $\sigma := (\mathsf{Sign}_1(\mathsf{sk}_m, m), (\sigma_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))_{u \prec m})$

- $\underline{\sigma := \mathsf{CSign}(\mathsf{sk}^*, m)}$
  1. Parse sk^* as $\mathsf{sk}^* := (k^*, (\sigma_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))_{u \prec m^*})$
  2. If $m^* \prec m$ output $\perp$
  3. Let v be the longest common prefix of m and m^*
  4. For each u with $v \prec u \preceq m$ compute $r_u := \mathsf{FEval}(k^*, u)$ and $(\mathsf{sk}_u, \mathsf{pk}_u) := \mathsf{Gen}_1(1^\lambda; r_u)$
  5. For each u with $v \prec u \prec m$ compute $\sigma_u := \mathsf{Sign}_1(\mathsf{sk}_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))$
  6. Output the tuple $\sigma := (\mathsf{Sign}_1(\mathsf{sk}_m, m), (\sigma_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))_{u \prec m})$

- $\underline{b := \mathsf{Ver}(\mathsf{pk}, m, \sigma)}$
  1. Parse σ as $(\sigma_m, (\sigma_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))_{u \prec m})$
  2. Output 1 iff $\mathsf{Ver}_1(\mathsf{pk}_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u), \sigma_u) = 1$ for all $u \prec m$ and $\mathsf{Ver}_1(\mathsf{pk}_m, m, \sigma_m) = 1$

Fig. 3. Construction of PCS from CPRF and OTS.

the real security experiment. In $\mathbf{Hybrid}_1$, the seed r_ϵ is replaced by a uniformly random seed. This hybrid is indistinguishable from the previous by security of th CPRF:

On receipt a prefix $m^* \in \{0,1\}^{<\ell}$ from the adversary, the reduction first sends a challenge query $x^* := \epsilon$ to the CPRF challenger and embeds the response y in place of r_ϵ to derive $(\mathsf{sk}_\epsilon, \mathsf{pk}_\epsilon) := \mathsf{Gen}_1(1^\lambda; r_\epsilon)$ and sets $\mathsf{pk} := (\mathsf{pk}_{\epsilon,\ell})$. It then requests a constrained key k^* for the set S_{m^*}. Next it queries its evaluation oracle on all prefixes $u \preceq m^*$ except for ϵ to receive r_u, which allows to derive the respective signing key pairs and compute the signatures required for producing the constrained signing key $sk^* := (k^*, (\sigma_u, (pk_{u\|0}, \mathsf{pk}_{u\|1}, u))_{u \prec m^*})$. When the adversary makes a signing query m, the reduction queries the evaluation oracle to receive r_u for all $u \preceq m$ except for ϵ and uses these seeds and r_ϵ just as in the honest protocol. Hence, if the adversary could distinguish the $\mathbf{Hybrid}_0$ from $\mathbf{Hybrid}_1$, then the reduction could distinguish whether the challenge y was pseudorandom or random, thus breaking security of CPRF.

In $\mathbf{Hybrid}_2$, not only r_ϵ, but also $r_{m_1^*}$, where m_1^* denotes the first bit of prefix m^*, is sampled uniformly at random instead of pseudorandomly. Again, the two hybrids can be proven indistinguishable by selective security of the CPRF. We proceed with $\mathbf{Hybrid}_3, \ldots, \mathbf{Hybrid}_{|m^*|}$ in analogous way, where in $\mathbf{Hybrid}_{|m^*|}$ all the seeds of prefixes of m^* are replaced by random seeds, hence the signing key pairs are all drawn independently, using uniformly random seeds.

We continue by a sequence of hybrids that step by step replace all the seeds in S_{m^*} that are computed throughout the security experiment in response to a signature query (note, these are only polynomially many) by randomly sampled seeds. Thus, in the final hybrid, when the adversary produces a forgery $(m_\mathsf{A}, \sigma_\mathsf{A})$, then it must hold $m_\mathsf{A} \in S_{m^*}$ and σ_A must contain a chain of signatures that verifies w.r.t. $\mathsf{pk} := (\mathsf{pk}_\epsilon, \ell)$, where pk_ϵ was sent to A by the reduction. The signatures in the chain could either be computed by the adversary or extracted from a previous signature query. However, since the adversary did not receive a signature for m_A, at least one signature in the chain must be fresh and since all signature keys associated with prefixes of m^* are independently sampled, this implies that the first such fresh signature gives a forgery for the OTS scheme. □

5 Trapdoor VDFs with Advanced Features

In this section, we define more advanced trapdoor VDFs, such as trapdoor-homomorphic and trapdoor-constrained VDFs. Due to the generality of our VDF constructions, we can achieve these advanced VDFs by simply replacing the underlying PRF from our constructions with a more advanced PRF, such as a key-homomorphic or constrained PRF. Hence, one can also consider these advanced VDFs as adding both a notion of delay and verifiability to the underlying PRFs.

5.1 Trapdoor-Homomorphic VDFs

We define the notion of trapdoor-homomorphic VDFs. These are trapdoor VDFs in which there is a homomorphism from the trapdoor key space to the output space. Before describing trapdoor-homomorphic VDFs, we first define the notion of key-homomorphism.

Definition 10 (Key-Homomorphic Functions). *A function family* $\mathcal{F}_\lambda = \{\mathsf{F}\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}\}$ *is* $(\oplus, \otimes)$-key-homomorphic *if* $(\mathcal{K}, \oplus)$ *and* $(\mathcal{Y}, \otimes)$ *are groups and for every* $k_1, k_2 \in \mathcal{K}$ *and* $x \in \mathcal{X}$*,*

$$F(k_1, x) \otimes F(k_2, x) = F(k_1 \oplus k_2, x).$$

We formally define trapdoor-homomorphic VDFs as follows.

Definition 11 (Trapdoor-Homomorphic VDFs). *A trapdoor VDF* $\mathsf{TVDF} = (\mathsf{Setup}, \mathsf{TEval}, \mathsf{Eval}, \mathsf{Ver})$ *with input and outputs spaces* $\mathcal{X}, \mathcal{Y}$ *and trapdoor key space* $\mathcal{K}$*, is a* trapdoor-homomorphic *VDF if* $(\mathcal{K}, \oplus)$ *and* $(\mathcal{Y}, \otimes)$ *are groups and for any* $\lambda, t \in \mathbb{N}$ *and* $x \in \mathcal{X}$*,*

$$\Pr\left[y_1 \otimes y_2 = y \;:\; \begin{array}{l} (\mathsf{ek}_1, \mathsf{vk}_1, \mathsf{td}_1) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ (\mathsf{ek}_2, \mathsf{vk}_2, \mathsf{td}_2) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ (y_1, \pi_1) \leftarrow \mathsf{TEval}(\mathsf{td}_1, x) \\ (y_2, \pi_2) \leftarrow \mathsf{TEval}(\mathsf{td}_2, x) \\ (y, \pi) \leftarrow \mathsf{TEval}(\mathsf{td}_1 \oplus \mathsf{td}_2, x) \end{array} \right] = 1 \ .$$

Constructing Trapdoor-Homomorphic VDFs. As a testament to the general framework of our VDF construction, it follows that in our basic VDF construction of Fig. 1, if we use a key-homomorphic PPRF instead of a standard PPRF in computing the VDF value, the resulting VDF is a trapdoor-homomorphic VDF. We formalize this with the following corollary, and refer to Corollary 4 for a more general corollary and a proof.

Corollary 2. *Let* $\mathcal{F}_\lambda = \{\mathsf{F}\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}\}$ *be a* $(\oplus, \otimes)$*-key-homomorphic puncturable PRF family. If we use* $\mathcal{F}_\lambda$ *for the PRF defined by the key* k_1 *inside the trapdoor VDF construction given in Fig. 1, then we obtain a trapdoor-homomorphic VDF.*

We require the underlying key-homomorphic PRF to have puncturing capabilities, and hence, we need to consider key-homomorphic "constrained" PRFs, instead of "plain" key-homomorphic PRFs. We can obtain such key-homomorphic constrained PRFs in the standard model from (R)LWE assumption, as shown in [6, 19].

We remark that (R)LWE based constructions achieve only approximate homomorphism, i.e., they are "almost" key-homomorphic PRFs, due to the underlying error term, and hence, they only allow for a bounded number of homomorphic operations. Therefore, instantiated with such key-homomorphic PRFs, our trapdoor-homomorphic VDF also inherits the same limitations, and becomes "almost" trapdoor-homomorphic VDF.

5.2 Trapdoor-Constrained VDFs

We define the notion of trapdoor-constrained VDFs. These are trapdoor VDFs in which keys can be constrained to allow the VDF evaluation on subsets of the domain.

Definition 12 (Trapdoor-Constrained VDFs). *A trapdoor VDF* $\mathsf{TVDF} = (\mathsf{Setup}, \mathsf{TEval}, \mathsf{Eval}, \mathsf{Ver})$ *with input and outputs spaces* $\mathcal{X}, \mathcal{Y}$ *and trapdoor key space* $\mathcal{K}$ *is a* trapdoor-constrained *VDF w.r.t. a family of subsets* $\mathcal{S}_\lambda$ *over* $\mathcal{X}$ *with a* constrained trapdoor key *space* $\mathcal{K}_\mathcal{S}$*, such that* $\mathcal{K}_\mathcal{S} \cap \mathcal{K} = \emptyset$*, if* TEval *accepts inputs from* $(\mathcal{K} \cup \mathcal{K}_\mathcal{S}) \times \mathcal{X}$ *and there exists a PPT algorithm* Constr *that, on input a key* $\mathsf{td} \in \mathcal{K}$ *and a set* $S \in \mathcal{S}_\lambda$*, outputs a* constrained trapdoor td_S *such that for any* $\lambda, t \in \mathbb{N}$ *and* $x \in S$*,*

$$\Pr\left[\begin{array}{l} y' = y \ \wedge \\ \mathsf{Ver}(\mathsf{vk}, x, y', \pi') = 1 \end{array} : \begin{array}{l} (\mathsf{ek}, \mathsf{vk}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda, t) \\ \mathsf{td}_S \leftarrow \mathsf{Constr}(\mathsf{td}, S) \\ (y, \pi) \leftarrow \mathsf{TEval}(\mathsf{td}, x) \\ (y', \pi') \leftarrow \mathsf{TEval}(\mathsf{td}_S, x) \end{array}\right] = 1 \ ,$$

and $\bot \leftarrow \mathsf{TEval}(\mathsf{td}_S, x')$*, for any* $x' \notin S$*.*

Constructing Trapdoor-Constrained VDFs. Similar to the trapdoor-homomorphic VDF given in Section 5.1, we can turn our trapdoor VDF given in Fig. 1 into a trapdoor-constrained VDF by replacing the underlying puncturable PRF with a constrained PRF. We formalize this with the following corollary, and refer to Corollary 4 for a more general corollary and a proof.

Corollary 3. *Let* $\mathcal{F}_\lambda = \{\mathsf{F} \colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}\}$ *be a constrained PRF family. If we use* $\mathcal{F}_\lambda$ *for the PRF defined by the key* k_1 *inside the trapdoor VDF construction from Fig. 1, then we obtain a trapdoor-constrained VDF.*

Depending on the constraining predicate, we can obtain constrained PRFs from different assumptions. We have prefix-constrained [35, 17] and constrained PRFs for t-CNF predicates from OWFs [26], for inner-product predicates from LWE [26], for P/poly and sets recognizable by Turing machines from (differing-input) obfuscation [26, 1].

5.3 VDFs with Homomorphic and Constrained Trapdoors

We define the notion of trapdoor homomorphic and constrained VDFs. These are trapdoor VDFs that combine the functionality of both trapdoor-homomorphic VDFs (see Section 5.1) and trapdoor-constrained VDFs (see Section 5.2). Hence, they have a homomorphism from the trapdoor key space to the output space and at the same time allow constraining the evaluation on a subset of the domain.

Definition 13 (Trapdoor-Homomorphic and Constrained VDFs). *A trapdoor VDF* $\mathsf{TVDF} = (\mathsf{Setup}, \mathsf{TEval}, \mathsf{Eval}, \mathsf{Ver})$ *is a* trapdoor homomorphic and constrained *VDF if it is both trapdoor-homomorphic VDF (satisfies Definition 11) and trapdoor-constrained VDF (satisfies Definition 12).*

Constructing Trapdoor-Homomorphic and Constrained VDFs. We can construct trapdoor-homomorphic and constrained VDFs by instantiating our trapdoor VDF construction from Fig. 1 with a key-homomorphic constrained PRF. Hence, we obtain the following corollary which subsumes Corollaries 2 and 3.

Corollary 4. *Let $\mathcal{F}_\lambda = \{\mathsf{F}\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}\}$ be a key-homomorphic constrained PRF family. If we use $\mathcal{F}_\lambda$ for the PRF defined by the key k_1 inside the trapdoor VDF construction from Fig. 1, then we obtain a trapdoor-homomorphic and constrained VDF.*

Proof. Let $\mathsf{TVDF} : \mathcal{X} \to \mathcal{Y}$ be the trapdoor VDF function given in Fig. 1, with its underlying trapdoor key space $\mathcal{K}$. It is easy to see that

- The evaluation (Eval) and trapdoor evaluation (TEval) algorithms are deterministic.
- The trapdoor key, input and output spaces $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ of the trapdoor VDF and the underlying puncturable PRF defined by the key k_1 are the same.
- The VDF output is of the form $y := \mathsf{PRF.FEval}(k_1, t\|x)$, and its proof is empty, i.e., $\pi := \bot$.
- The trapdoor key is defined as $\mathsf{td} := k_1$.

Hence, we can define the trapdoor evaluation and trapdoor constraining algorithms as follows:

- $y := \mathsf{TVDF.TEval}(\mathsf{td}, t\|x) = \mathsf{PRF.FEval}(\mathsf{td}, t\|x)$;
- $\mathsf{td}_S \leftarrow \mathsf{TVDF.Constr}(\mathsf{td}, S) = \mathsf{PRF.Constr}(\mathsf{td}, S)$.

Moreover, due to the key-homomorphism of the PRF, we get that

$$\mathsf{TVDF.TEval}(\mathsf{td}_1, t\|x) \otimes \mathsf{TVDF.TEval}(\mathsf{td}_1, t\|x) = \mathsf{TVDF.TEval}(\mathsf{td}_1 \oplus \mathsf{td}_2, t\|x),$$

and the pseudorandomness of the VDF is preserved due to the pseudorandomness of the PRF. Therefore, if the underlying PRF is a key-homomorphic constrained PRF, then TVDF is a trapdoor-homomorphic and constrained VDF. □

As described in Section 5.1, we have such key-homomorphic constrained PRFs from (R)LWE assumption [6, 19], which we can use to instantiate our construction in order to obtain "almost" trapdoor-homomorphic and constrained VDFs (without preprocessing).

Acknowledgments

We would like to thank Gennaro Avitabile for pointing us to the utility of witness maps in our context, and an anonymous reviewer from TCC'25 for bringing to our attention the talk [12]. Hamza Abusalah and Dario Fiore are supported by the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Unions Horizon 2020 research and

innovation programme (Grant agreement No. 101001283), partially supported by projects PRODIGY (TED2021-132464B- I00), ESPADA (PID2022-142290OB-I00) and CEX2024-001471-M funded by MCIN/AEI/10.13039/501100011033/. Erkan Tairi is supported in part by the France 2030 ANR-22-PECY-003 SecureCompute Project.

References

1. Abusalah, H., Fuchsbauer, G., Pietrzak, K.: Constrained PRFs for unbounded inputs. In: CT-RSA 2016 36
2. Ahrens, K., Zumbrägel, J.: DEFEND: towards verifiable delay functions from endomorphism rings. IACR Cryptol. ePrint Arch. 2, 3, 12
3. Applebaum, B., Ishai, Y., Kushilevitz, E.: Cryptography in NC^0. In: 45th FOCS 4
4. Armknecht, F., Barman, L., Bohli, J.M., Karame, G.O.: Mirror: Enabling proofs of data replication and retrievability in the cloud. In: USENIX Security 2016 2
5. Arun, A., Bonneau, J., Clark, J.: Short-lived zero-knowledge proofs and signatures. In: ASIACRYPT 2022, Part III 7
6. Banerjee, A., Fuchsbauer, G., Peikert, C., Pietrzak, K., Stevens, S.: Key-homomorphic constrained pseudorandom functions. In: TCC 2015, Part II 11, 35, 37
7. Bellare, M., Fuchsbauer, G.: Policy-based signatures. In: PKC 2014 11, 16
8. Biryukov, A., Fisch, B., Herold, G., Khovratovich, D., Leurent, G., Naya-Plasencia, M., Wesolowski, B.: Cryptanalysis of algebraic verifiable delay functions. In: Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part III 12
9. Bitansky, N., Choudhuri, A.R., Holmgren, J., Kamath, C., Lombardi, A., Paneth, O., Rothblum, R.D.: PPAD is as hard as LWE and iterated squaring. In: TCC 2022, Part II 2, 3
10. Bitansky, N., Garg, R.: Succinct randomized encodings from laconic function evaluation, faster and simpler. In: Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part VII 4
11. Bitansky, N., Goldwasser, S., Jain, A., Paneth, O., Vaikuntanathan, V., Waters, B.: Time-lock puzzles from randomized encodings. In: ITCS 2016 4, 5, 6, 7, 13, 17, 27
12. Bitansky, N., Goldwasser, S., Jain, A., Paneth, O., Vaikuntanathan, V., Waters, B.: Alternate vdf constructions. MIT VDF Day 2019, youtube video, accessed on September 29, 2025 at https://www.youtube.com/watch?v=ckIYmtY3KAw 13, 37
13. Bitansky, N., Paneth, O., Wichs, D.: Perfect structure on the edge of chaos - trapdoor permutations from indistinguishability obfuscation. In: TCC 2016-A, Part I 7, 27
14. Block, A.R., Holmgren, J., Rosen, A., Rothblum, R.D., Soni, P.: Time- and space-efficient arguments from groups of unknown order. In: CRYPTO 2021, Part IV 2, 3, 6
15. Boneh, D., Bonneau, J., Bünz, B., Fisch, B.: Verifiable delay functions. In: CRYPTO 2018, Part I 2, 3, 4, 5, 6, 12, 18, 20

16. Boneh, D., Bonneau, J., Bünz, B., Fisch, B.: Verifiable delay functions. IACR Cryptol. ePrint Arch. 4
17. Boneh, D., Waters, B.: Constrained pseudorandom functions and their applications. In: ASIACRYPT 2013, Part II 11, 15, 36
18. Boyle, E., Goldwasser, S., Ivan, I.: Functional signatures and pseudorandom functions. In: PKC 2014 11, 15
19. Brakerski, Z., Vaikuntanathan, V.: Constrained key-homomorphic PRFs from standard lattice assumptions - or: How to secretly embed a circuit in your PRF. In: TCC 2015, Part II 11, 35, 37
20. Canetti, R., Lin, H., Tessaro, S., Vaikuntanathan, V.: Obfuscation of probabilistic circuits and applications. In: TCC 2015, Part II 8
21. Chakraborty, S., Prabhakaran, M., Wichs, D.: Witness maps and applications. In: PKC 2020, Part I 7, 11, 30, 32
22. Chávez-Saab, J., Rodríguez-Henríquez, F., Tibouchi, M.: Verifiable isogeny walks: Towards an isogeny-based postquantum VDF. In: Selected Areas in Cryptography - 28th International Conference, SAC 2021, Virtual Event, September 29 - October 1, 2021, Revised Selected Papers 2, 3, 12
23. Choudhuri, A.R., Hubacek, P., Kamath, C., Pietrzak, K., Rosen, A., Rothblum, G.N.: PPAD-hardness via iterated squaring modulo a composite. Cryptology ePrint Archive, Report 2019/667, https://eprint.iacr.org/2019/667 2
24. Cini, V., Lai, R.W.F., Malavolta, G.: Lattice-based succinct arguments from vanishing polynomials - (extended abstract). In: CRYPTO 2023, Part II 3
25. Cohen, B., Pietrzak, K.: The chia network blockchain, accessed on September 29, 2024 at https://www.chia.net/wp-content/uploads/2022/07/ChiaGreenPaper.pdf 2
26. Davidson, A., Katsumata, S., Nishimaki, R., Yamada, S., Yamakawa, T.: Adaptively secure constrained pseudorandom functions in the standard model. In: CRYPTO 2020, Part I 36
27. De Feo, L., Masson, S., Petit, C., Sanso, A.: Verifiable delay functions from supersingular isogenies and pairings. In: ASIACRYPT 2019, Part I 2, 3, 12
28. Döttling, N., Garg, S., Malavolta, G., Vasudevan, P.N.: Tight verifiable delay functions. In: SCN 20 2, 3, 6, 13
29. Dujmovic, J., Garg, R., Malavolta, G.: Time-lock puzzles with efficient batch solving. In: Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part II 5
30. Ephraim, N., Freitag, C., Komargodski, I., Pass, R.: Continuous verifiable delay functions. In: EUROCRYPT 2020, Part III 2, 3
31. Ephraim, N., Freitag, C., Komargodski, I., Pass, R.: SPARKs: Succinct parallelizable arguments of knowledge. In: EUROCRYPT 2020, Part I 2, 3
32. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: CRYPTO'86 3
33. Freitag, C., Pass, R., Sirkin, N.: Parallelizable delegation from LWE. In: TCC 2022, Part II 2, 3, 4, 5, 6
34. Garg, S., Gentry, C., Halevi, S., Raykova, M., Sahai, A., Waters, B.: Candidate indistinguishability obfuscation and functional encryption for all circuits. In: 54th FOCS 14
35. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions (extended abstract). In: 25th FOCS 7, 12, 32, 36
36. Goyal, R., Kim, S., Manohar, N., Waters, B., Wu, D.J.: Watermarking public-key cryptographic primitives. In: CRYPTO 2019, Part III 16

37. Hoffmann, C., Hubácek, P., Kamath, C., Klein, K., Pietrzak, K.: Practical statistically-sound proofs of exponentiation in any group. In: CRYPTO 2022, Part II 2, 3
38. Hoffmann, C., Hubácek, P., Kamath, C., Krnák, T.: (Verifiable) delay functions from lucas sequences. In: TCC 2023, Part IV 2, 3
39. Jaques, S., Montgomery, H., Rosie, R., Roy, A.: Time-release cryptography from minimal circuit assumptions. In: Progress in Cryptology – INDOCRYPT 2021 2, 3
40. Katz, J., Loss, J., Xu, J.: On the security of time-lock puzzles and timed commitments. In: TCC 2020, Part III 3
41. Khovratovich, D., Maller, M., Tiwari, P.R.: Minroot: Candidate sequential function for ethereum VDF. IACR Cryptol. ePrint Arch. 2, 3, 12
42. Kiayias, A., Papadopoulos, S., Triandopoulos, N., Zacharias, T.: Delegatable pseudorandom functions and applications. In: ACM CCS 2013 11, 15
43. Lai, R.W.F., Malavolta, G.: Lattice-based timed cryptography. In: CRYPTO 2023, Part V 3
44. Lamport, L.: Constructing digital signatures from a one-way function. Technical Report SRI-CSL-98, SRI International Computer Science Laboratory 11, 32
45. Landerreche, E., Stevens, M., Schaffner, C.: Non-interactive cryptographic timestamping based on verifiable delay functions. In: FC 2020 2
46. Lenstra, A.K., Wesolowski, B.: Trustworthy public randomness with sloth, unicorn, and trx. Int. J. Appl. Cryptogr. 2
47. Lombardi, A., Vaikuntanathan, V.: Fiat-shamir for repeated squaring with applications to PPAD-hardness and VDFs. In: CRYPTO 2020, Part III 2, 3
48. Mahmoody, M., Moran, T., Vadhan, S.P.: Publicly verifiable proofs of sequential work. In: ITCS 2013 3
49. Mahmoody, M., Smith, C., Wu, D.J.: Can verifiable delay functions be based on random oracles? In: ICALP 2020 12
50. Malavolta, G., Thyagarajan, S.A.K.: Homomorphic time-lock puzzles and applications. In: CRYPTO 2019, Part I 5
51. Peikert, C., Tang, Y.: Cryptanalysis of lattice-based sequentiality assumptions and proofs of sequential work. In: Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part V 3
52. Pietrzak, K.: Simple verifiable delay functions. In: ITCS 2019 2, 3, 5, 6
53. Rivest, R.L., Shamir, A., Wagner, D.A.: Time-lock puzzles and timed-release crypto. Tech. rep., USA 3
54. Rivest, R.L., Shamir, A., Wagner, D.A.: Time-lock puzzles and timed-release crypto. Tech. rep., USA 4, 5
55. Rotem, L., Segev, G., Shahaf, I.: Generic-group delay functions require hidden-order groups. In: EUROCRYPT 2020, Part III 13
56. Sahai, A., Waters, B.: How to use indistinguishability obfuscation: deniable encryption, and more. In: 46th ACM STOC 8, 9, 10, 16
57. StarkWare: Veedo, accessed on October 1st, 2024 at https://medium.com/starkware/presenting-veedo-e4bbff77c7ae 2, 3, 12
58. Waters, B., Wu, D.J.: Adaptively-sound succinct arguments for NP from indistinguishability obfuscation. In: STOC 6
59. Waters, B., Wu, D.J.: Adaptively-sound succinct arguments for NP from indistinguishability obfuscation. Cryptology ePrint Archive, Paper 2024/165 10
60. Waters, B., Zhandry, M.: Adaptive security in snargs via io and lossy functions. In: CRYPTO (10) 6

61. Waters, B., Zhandry, M.: Adaptive security in SNARGs via iO and lossy functions. Cryptology ePrint Archive, Paper 2024/254 10
62. Wesolowski, B.: Efficient verifiable delay functions. In: EUROCRYPT 2019, Part III 2, 3, 5, 6, 18

A Proof of Sequentiality in Theorem 2

In this section we provide the proof of claim of sequentiality in Theorem 2.

Proof. We proceed by a hybrid argument consisting of six hybrids $\mathbf{Hybrid}_0$, $\cdots$, $\mathbf{Hybrid}_5$, which we describe below.

– $\mathbf{Hybrid}_0$ is the distribution of the "real" sequentiality experiment. Hence it is generated according to the protocol description in Fig. 2 and the experiment outputs 1 if and only if the adversary breaks sequentiality of VDF_2 without preprocessing. Note again that assuming perfect correctness of iO and TLP, the winning condition simplifies to "$s^*_{\mathsf{A}} = \mathsf{FEval}(k_1, t\|x^*)$".

$\mathbf{Hybrid}_0(1^\lambda, t)$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample a PCS key-pair $(\mathsf{pk}, \mathsf{msk}) \leftarrow \mathsf{Gen}(1^\lambda)$
3. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1,k_2,\mathsf{msk}]}(\cdot))$ and set $\mathsf{vk} := (\mathsf{ek}, \mathsf{pk})$, where

   $(z, \pi, \sigma) := \mathsf{Sample}_{[t,k_1,k_2,\mathsf{msk}]}(x)$

   (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1, t\|x)$
   (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
   (c) Compute the signatures $\pi := p_\lambda(s)$ and $\sigma := \mathsf{Sign}(\mathsf{msk}, x\|t\|z)$
   (d) Output (z, π, σ)
4. Sample a random challenge $x^* \leftarrow \mathcal{X}_\lambda$
5. Run A on $(\mathsf{ek}, \mathsf{vk}, x^*)$ to obtain s^*_{A}
6. Output 1 if and only if $s^*_{\mathsf{A}} = \mathsf{FEval}(k_1, t\|x^*)$

– In $\mathbf{Hybrid}_1$, we puncture the key k_1 at the challenge $\{t\|x^*\}$. The resulting distribution is indistinguishable from $\mathbf{Hybrid}_0$ thanks to preservation of PPRF functionality under puncturing and IO security.
– $\mathbf{Hybrid}_2$ is similar to $\mathbf{Hybrid}_1$ except that the challenge solution s^* is sampled at random. The resulting distribution is indistinguishable from $\mathbf{Hybrid}_0$ thanks to pseudorandomness of PPRF at punctured points.

Hybrid$_1(1^\lambda, t)$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample a PCS key-pair $(\mathsf{pk}, \mathsf{msk}) \leftarrow \mathsf{Gen}(1^\lambda)$
3. Sample a random challenge $x^* \leftarrow \mathcal{X}_\lambda$, and a punctured key $k_1^* \leftarrow \mathsf{Puncture}(k_1, \{t\|x^*\})$
4. Pre-compute the *challenge* solution-puzzle pair with a signature:
   (a) Generate a pseudorandom TLP solution $s^* := \mathsf{FEval}(k_1, t\|x^*)$
   (b) Use s^* and *pseudorandom* coins to generate a TLP puzzle $z^* := \mathsf{PGen}(s^*, t; \mathsf{FEval}(k_2, t\|x^*))$
   (c) Compute the challenge signatures $\pi^* := p_\lambda(s^*)$ and $\sigma^* := \mathsf{Sign}(\mathsf{msk}, x^*\|\pi^*\|z^*)$
5. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1^*,k_2,\mathsf{msk},x^*,z^*,\pi^*,\sigma^*]}(\cdot))$ and set $\mathsf{vk} := \mathsf{ek}$, where

   $(z, \pi, \sigma) := \mathsf{Sample}_{[t,k_1^*,k_2,\mathsf{msk},x^*,z^*,\pi^*]}(x)$

   - If $x = x^*$ then output (z^*, π^*, σ^*)
   - Else
     (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1^*, t\|x)$
     (b) Use s and *pseudorandom* coins to generate a TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2, t\|x))$
     (c) Compute the signatures $\pi := p_\lambda(s)$ and $\sigma := \mathsf{Sign}(\mathsf{msk}, x\|\pi\|z)$
     (d) Output (z, π, σ)
6. Run A on $(\mathsf{ek}, \mathsf{vk}, x^*)$ to obtain s_A^*
7. Output 1 if and only if $s_\mathsf{A}^* = s^*$

Hybrid$_2(1^\lambda, t)$

$\vdots$

4. Pre-compute the *challenge* solution-puzzle pair with a signature:
   (a) Sample a *random* TLP solution $s^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Generate the TLP puzzle $z^* := \mathsf{PGen}(s^*, t; \mathsf{FEval}(k_2, t\|x^*))$ from s^* using *pseudorandom* coins
   (c) Compute the signatures $\pi^* := p_\lambda(s^*)$ and $\sigma := \mathsf{Sign}(\mathsf{msk}, x^*\|\pi^*\|z^*)$

$\vdots$

7. Output 1 if and only if $s_\mathsf{A}^* = s^*$

– In $\mathbf{Hybrid}_3$, we puncture the key k_2 at $\{t\|x^*\}$. Looking ahead, the purpose of this step is to undo the "derandomisation" carried out in Setup for generating the TLP puzzle. As for $\mathbf{Hybrid}_1$, this change is indistinguishable thanks to preservation of PPRF functionality under puncturing and IO security.

$\mathbf{Hybrid}_3(1^\lambda, t)$

1. Sample two PPRF keys $k_1, k_2 \leftarrow \mathsf{KGen}(1^\lambda)$
2. Sample a PCS key-pair $(\mathsf{pk}, \mathsf{msk}) \leftarrow \mathsf{Gen}(1^\lambda)$
3. Sample a random challenge $x^* \leftarrow \mathcal{X}_\lambda$, and a punctured key $k_1^* \leftarrow \mathsf{Puncture}(k_1, \{t\|x^*\})$
4. Pre-compute the *challenge* solution-puzzle pair with a signature:
   (a) Sample a *random* TLP solution $s^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Sample a punctured key $k_2^* \leftarrow \mathsf{Puncture}(k_2, \{t\|x^*\})$
   (c) Use s^* and *pseudorandom* coins to generate the TLP puzzle $z^* := \mathsf{PGen}(s^*, t; \mathsf{FEval}(k_2, t\|x^*))$
   (d) Compute the signatures $\pi^* := p_\lambda(s^*)$ and $\sigma^* := \mathsf{Sign}(\mathsf{msk}, x^*\|\pi^*\|z^*)$
5. Sample $\mathsf{ek} \leftarrow \mathsf{iO}(\mathsf{Sample}_{[t,k_1^*,k_2^*,\mathsf{msk},x^*,z^*,\pi^*,\sigma^*]}(\cdot))$ and set $\mathsf{vk} := \mathsf{ek}$, where

   $(z, \pi, \sigma) := \mathsf{Sample}_{[t,k_1^*,k_2^*,\mathsf{msk},x^*,z^*,\pi^*,\sigma^*]}(x)$

   - If $x = x^*$ then output (z^*, π^*, σ^*)
   - Else
     (a) Generate a pseudorandom TLP solution $s := \mathsf{FEval}(k_1^*, t\|x)$
     (b) Use s and *pseudorandom* coins to generate the TLP puzzle $z := \mathsf{PGen}(s, t; \mathsf{FEval}(k_2^*, t\|x))$
     (c) Compute the signatures $\pi := p_\lambda(s)$ and $\sigma := \mathsf{Sign}(\mathsf{msk}, x\|\pi\|z)$
     (d) Output (z, π, σ)
6. Run A on $(\mathsf{ek}, \mathsf{vk}, x^*)$ to obtain s_A^*
7. Output 1 if and only if $s_\mathsf{A}^* = s^*$

– In $\mathbf{Hybrid}_4$, we switch the random coins used for generating the challenge TLP puzzle from pseudorandom to truly random. This is necessary to invoke sequentiality of TLP in the next step. As for $\mathbf{Hybrid}_2$, this change is indistinguishable thanks to pseudorandomness of PPRF at punctured points.

– Finally, in $\mathbf{Hybrid}_5$, we decouple the signature from the puzzle: a *second* solution s_2^* is used to generate the challenge puzzle, but the adversary wins if it still outputs s_1^* (which is the first solution). However, this change is indistinguishable to *bounded-depth* adversaries thanks to TLP sequentiality without preprocessing, and thus the probabilities that a bounded-depth adversary outputs s_1^* in $\mathbf{Hybrid}_4$ and $\mathbf{Hybrid}_5$ are negligibly close.

Hybrid$_4(1^\lambda, t)$

$\vdots$

4. Pre-compute the *challenge* solution-puzzle pair with a signature:
   (a) Sample a *random* TLP solution $s^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Sample a punctured key $k_2^* \leftarrow \mathsf{Puncture}(k_2, \{t \| x^*\})$
   (c) Sample random coins $r^* \leftarrow \{0,1\}^{\mathrm{poly}(\lambda)}$
   (d) Use s_1^* and *random* coins to generate the TLP puzzle $z^* := \mathsf{PGen}(s^*, t; r^*)$
   (e) Compute the signatures $\pi^* := p_\lambda(s^*)$ and $\sigma^* := \mathsf{Sign}(\mathsf{msk}, x^* \| \pi^* \| z^*)$

$\vdots$

7. Output 1 if and only if $s_{\mathsf{A}}^* = s^*$

The probability that experiment **Hybrid**$_5(1^\lambda, t)$ outputs 1 can be bounded by the one-wayness of $\mathcal{P}$ since the only the signature π^* has any information about s_1^*. Since the depth of all the reductions depends on t only by a $\log(t)$ factor, the gap of VDF is sequential with $O(\epsilon)$ gap.

□

Hybrid$_5(1^\lambda, t)$

$\vdots$

4. Pre-compute the *challenge* solution-puzzle pair with a signature:
   (a) Sample a *random* TLP solution $s_1^*, s_2^* \leftarrow \{0,1\}^{m(\lambda)}$
   (b) Sample a punctured key $k_2^* \leftarrow \mathsf{Puncture}(k_2, \{t \| x^*\})$
   (c) Sample random coins $r^* \leftarrow \{0,1\}^{\mathrm{poly}(\lambda)}$
   (d) Use s_2^* and *random* coins to generate the TLP puzzle $z_2^* := \mathsf{PGen}(s_2^*, t; r^*)$
   (e) Compute the signatures $\pi^* := p_\lambda(s_1^*)$ and $\sigma^* := \mathsf{Sign}(\mathsf{msk}, x^* \| \pi^* \| z_2^*)$

$\vdots$

7. Output 1 if and only if $s_{\mathsf{A}}^* = s_1^*$